

Invertible Normalizing Flows for Generative Image Modeling: Executable Methods and Results Log

Matt Tivnan*

September 1, 2026

Abstract

This document is the executable methods-and-results log for the invertible normalizing flow project. Each numbered step of the pipeline is one `step*.py` script paired with one YAML config in `configs/` and one output folder under `outputs/`; the tables and figures below are included directly from those output folders, so the compiled PDF is an exact record of what was run and what it produced. Step 1 (this version) builds PyTorch datasets and dataloaders with deterministic train/val/test splits for the standard generative-image-modeling benchmarks: MNIST, FashionMNIST, CIFAR-10/100, SVHN, the MedMNIST 2D collection, CelebA, ImageNet, AFHQ, FFHQ, CelebA-HQ, and LSUN bedroom.

1 Execution environment

- Host: `axis03`; container: `recon-dev` (image `localhost:5000/recon:acr-20260819-v1`; PyTorch 2.11.0+cu128, torchvision 0.26.0, CUDA 12.4, 2 GPUs).
- Repository mounted at `/dev_ws/invertible_normalizing_flow_20260831` inside the container (shared workspace `/home/staticct/recon-dev/workspace` on the host).
- Datasets live on the `axis03` data drive at `/data/image_benchmarks`, mounted into the container at the same path. This is a shared local repository of public benchmarks, reused across projects; nothing dataset-sized is stored in git.
- All steps are run as `docker exec recon-dev python /dev_ws/invertible_normalizing_flow_20260831/s`
- Exact package versions, the resolved config, and the command line for the run shown in this PDF are recorded in `outputs/step1_datasets/data/provenance.json`.

2 Step 1: Datasets and dataloaders

2.1 Methods

- Code: `step1_datasets.py` (repository root). Config: `configs/step1_datasets.yml`. Outputs: `outputs/step1_datasets/{data,figures}/`.
- Command executed for this document: `python step1_datasets.py` (runs every dataset enabled in the config).

*Repository: github.com/tivnanmatt/invertible_normalizing_flow_20260831

- The script exposes a library API used by later steps: `build_dataset(name, cfg)` returns a bundle of three `torch.utils.data.Dataset` splits plus metadata, and `get_dataloaders(bundle, cfg)` wraps them in `DataLoaders` (batch size, workers, pinning from the config).
- Auto-download datasets: MNIST, FashionMNIST, CIFAR-10, CIFAR-100, SVHN (torchvision); the MedMNIST 2D collection (`medmnist` package, official train/val/test); Imagenette (fast.ai 10-class ImageNet subset, used as a fast stand-in for ImageNet); AFHQ (StarGAN-v2 release, fetched from the official Dropbox link); CelebA (torchvision Google-Drive download, quota permitting).
- Manual-download datasets (access terms or hosting): ImageNet (ILSVRC2012), FFHQ, CelebA-HQ, LSUN bedroom. When absent, the script records them as *unavailable* together with exact acquisition instructions (Section 2.4) instead of failing.
- Train/val/test policy (deterministic, seeded from the config): official splits are used whenever they exist (MedMNIST, CelebA); datasets with only train/test get a validation set carved from train (10%); datasets whose public split is train/val only (Imagenette, ImageNet, AFHQ) use the official val as *test* and carve val from train; single-pool datasets (FFHQ, CelebA-HQ) are split 80/10/10.
- Transforms: `ToTensor` to $[0, 1]$ floats only; variable-size natural-image datasets additionally get `Resize+CenterCrop` to the per-dataset `image_size` in the config. No normalization or dequantization is baked in here – for flow training those belong to the model step so the data step stays lossless.
- Per dataset the run itself is the test: all three splits are instantiated, real batches are pulled through the dataloaders, per-channel mean/std are computed over up to 20 train batches, and three figures are written (sample grid per split, pixel-intensity histogram, class distribution where integer labels exist).
- Machine-readable outputs: one JSON per dataset, `summary.json`, `summary.csv`, `provenance.json`, and the L^AT_EX fragments (`summary_table.tex`, `figures_step1.tex`, `unavailable_step1.tex`) that are included verbatim below.

2.2 Configuration (verbatim)

```
# Config for step1_datasets.py -- datasets & dataloaders.
#
# data_root is the shared benchmark-dataset repository on the axis03 data
# drive, mounted into the recon-dev container at the same path
# (/data/image_benchmarks). Datasets download/live there once and are reused
# across projects; nothing dataset-sized lands in the git repo.

seed: 20260831
data_root: /data/image_benchmarks
output_root: outputs/step1_datasets

loader:
  batch_size: 64
  num_workers: 4
  pin_memory: true
```

```

split:
  # carved from the official train pool when no official val split exists
  val_fraction: 0.10
  # only used for single-pool datasets with no official splits (ffhq, celeba_hq)
  test_fraction: 0.10

figures:
  samples_per_split: 8      # images per row in the sample-grid figure
  stats_max_batches: 20    # batches used for per-channel mean/std

datasets:
  # ---- small standard benchmarks (auto-download) ----
  mnist:          {enabled: true}
  fashion_mnist: {enabled: true}
  cifar10:        {enabled: true}
  cifar100:       {enabled: true}
  svhn:           {enabled: true}

  # ---- MedMNIST 2D collection (auto-download) ----
  # All 12 2D members are supported; the run set below spans the modalities
  # (pathology, blood smear, dermoscopy, X-ray, ultrasound, fundus, abdominal
  # CT). octmnist/tissuemnist/chestmnist are large; enable by adding flags.
  medmnist:
    enabled: true
    size: 28
    flags:
      - pathmnist
      - bloodmnist
      - dermamnist
      - pneumoniamnist
      - breastmnist
      - retinamnist
      - organamnist

  # ---- faces / natural images ----
  celeba:          # aligned CelebA; auto-download tried, gdrive quota permitting
    enabled: true
    image_size: 64
  imagenette:      # fast.ai 10-class ImageNet subset; auto-download stand-in
    enabled: true  # for full ImageNet in quick experiments
    variant: 160px
    image_size: 64
  imagenet:        # full ILSVRC2012; manual download (access terms)
    enabled: true
    image_size: 64
  afhq:           # animal faces (cat/dog/wild), StarGAN-v2 release
    enabled: true
    version: v1
    image_size: 64
    attempt_download: true
  ffhq:           # 70k faces; manual download (NVLabs)
    enabled: true
    image_size: 128
  celeba_hq:       # 30k faces; manual download

```

```

enabled: true
image_size: 256
lsun_bedroom: # LSUN lmbd; manual download
enabled: true
image_size: 64

```

2.3 Results: dataset summary

dataset	status	train	val	test	image	classes
mnist	ok	54000	6000	10000	$1 \times 28 \times 28$	10
fashion_mnist	ok	54000	6000	10000	$1 \times 28 \times 28$	10
cifar10	ok	45000	5000	10000	$3 \times 32 \times 32$	10
cifar100	ok	45000	5000	10000	$3 \times 32 \times 32$	100
svhn	ok	65931	7326	26032	$3 \times 32 \times 32$	10
medmnist_pathmnist	ok	89996	10004	7180	$3 \times 28 \times 28$	9
medmnist_bloodmnist	ok	11959	1712	3421	$3 \times 28 \times 28$	8
medmnist_dermamnist	ok	7007	1003	2005	$3 \times 28 \times 28$	7
medmnist_pneumoniamnist	ok	4708	524	624	$1 \times 28 \times 28$	2
medmnist_breastmnist	ok	546	78	156	$1 \times 28 \times 28$	2
medmnist_retinamnist	ok	1080	120	400	$3 \times 28 \times 28$	5
medmnist_organamnist	ok	34561	6491	17778	$1 \times 28 \times 28$	11
celeba	unavailable	—	—	—	—	—
imagenette	ok	8522	947	3925	$3 \times 64 \times 64$	10
imagenet	unavailable	—	—	—	—	—
afhq	ok	13167	1463	1500	$3 \times 64 \times 64$	3
ffhq	unavailable	—	—	—	—	—
celeba_hq	unavailable	—	—	—	—	—
lsun_bedroom	unavailable	—	—	—	—	—

Split sizes are reported after the split policy above; *image* is the tensor shape $C \times H \times W$ delivered by the dataloaders; *classes* is the number of integer classes (“—” for unconditional / multi-label datasets).

2.4 Results: unavailable datasets

- **celeba**: CelebA auto-download failed (Google Drive quota is a known issue: RuntimeError). Manually place `img_align_celeba/` plus the `list_*.txt` / `identity_*` files under `data/celeba/celeba/` (see `torchvision.datasets.CelebA` docs), then re-run.
- **imagenet**: ImageNet (ILSVRC2012) requires manual download (terms of access): obtain from <https://image-net.org>, extract to `data/imagenet/train/<wnid>/*.JPEG` and `data/imagenet/val/<wnid>/*.JPEG`, then re-run.
- **ffhq**: FFHQ requires manual download: place images (e.g. `thumbnails128x128` or `images1024x1024` from github.com/NVlabs/ffhq-dataset) under `data/ffhq/`
- **celeba_hq**: CelebA-HQ requires manual download: place the 30k CelebA-HQ images (e.g. `celeba_hq_256` from the progressive-GAN release) under `data/celeba_hq/`

- **lsun_bedroom**: LSUN bedroom requires manual download: fetch `bedroom_train_lmdb / bedroom_val_lmdb` with `github.com/fyu/lsun download.py` into `data/lsun/` (also needs ‘`pip install lmdb`’).

2.5 Results: figures

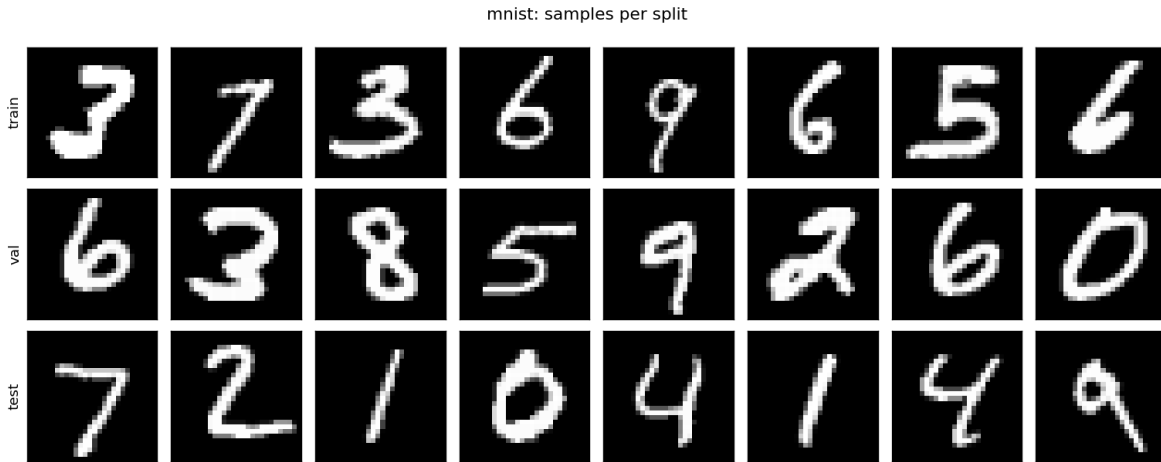


Figure 1: **mnist** (samples). Source: `torchvision.datasets.MNIST` (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_samples.png`, produced by `step1_datasets.py`.

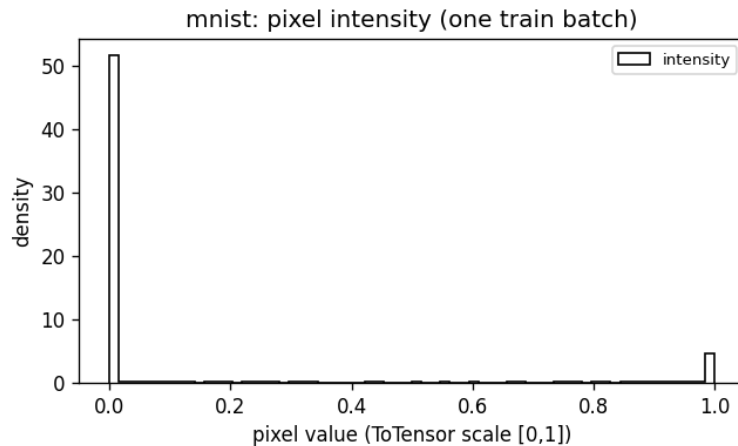


Figure 2: **mnist** (hist). Source: `torchvision.datasets.MNIST` (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_pixel_hist.png`, produced by `step1_datasets.py`.

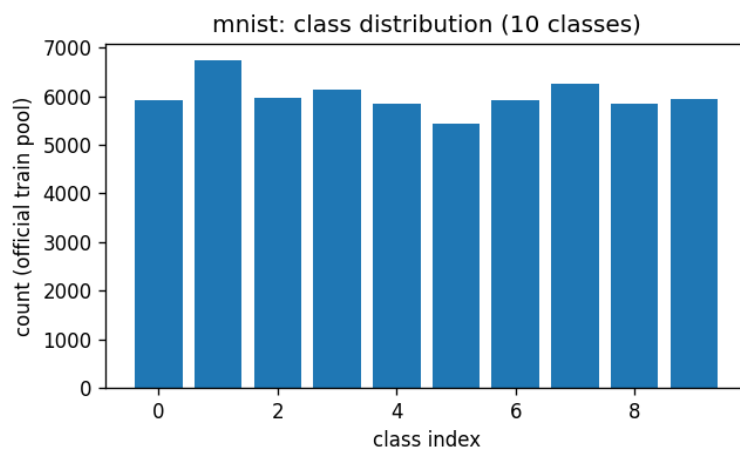


Figure 3: **mnist** (dist). Source: torchvision.datasets.MNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/mnist_class_dist.png, produced by step1_datasets.py.

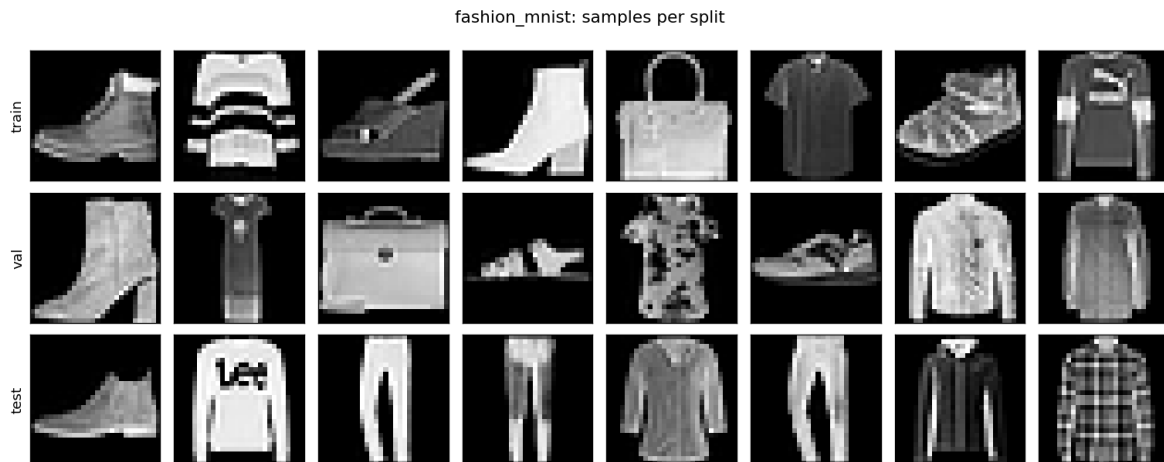


Figure 4: **fashion_mnist** (samples). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/fashion_mnist_samples.png, produced by step1_datasets.py.

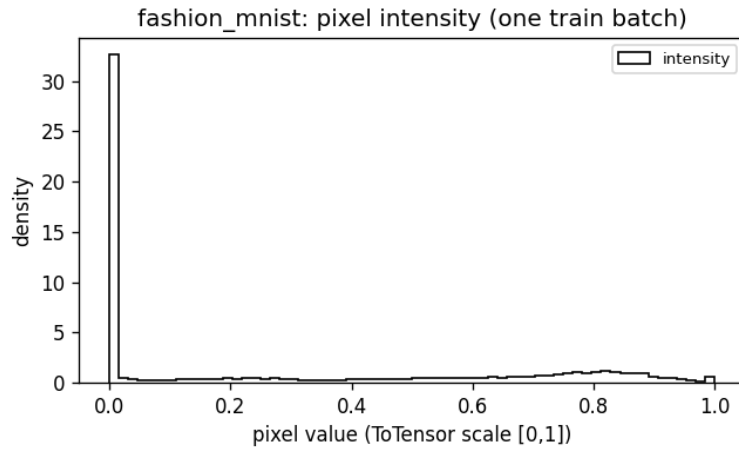


Figure 5: **fashion_mnist** (hist). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/fashion_mnist_pixel_hist.png, produced by step1_datasets.py.



Figure 6: **fashion_mnist** (dist). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/fashion_mnist_class_dist.png, produced by step1_datasets.py.

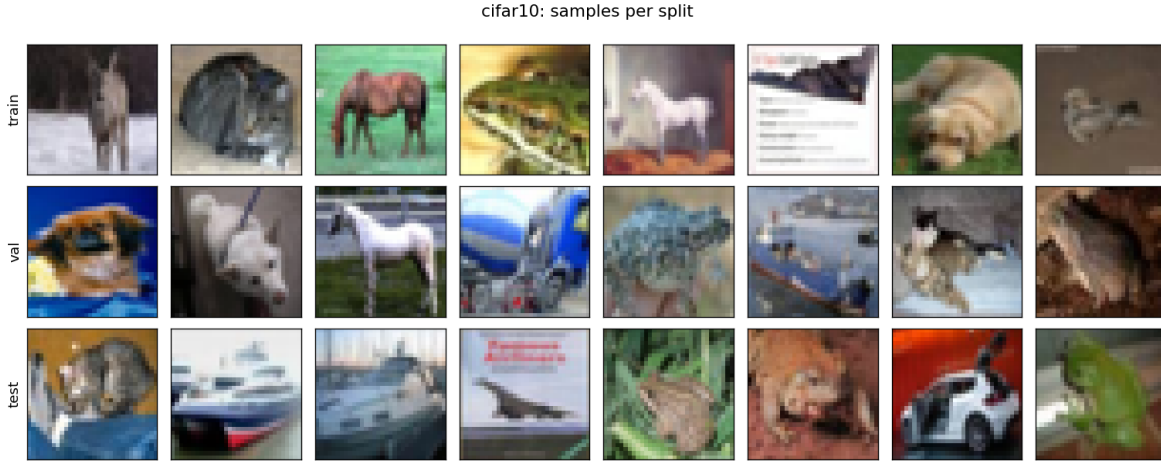


Figure 7: **cifar10** (samples). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar10_samples.png, produced by step1_datasets.py.

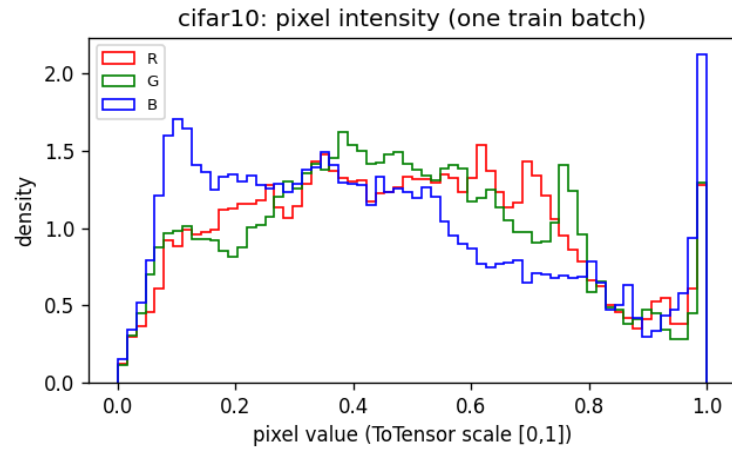


Figure 8: **cifar10** (hist). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar10_pixel_hist.png, produced by step1_datasets.py.

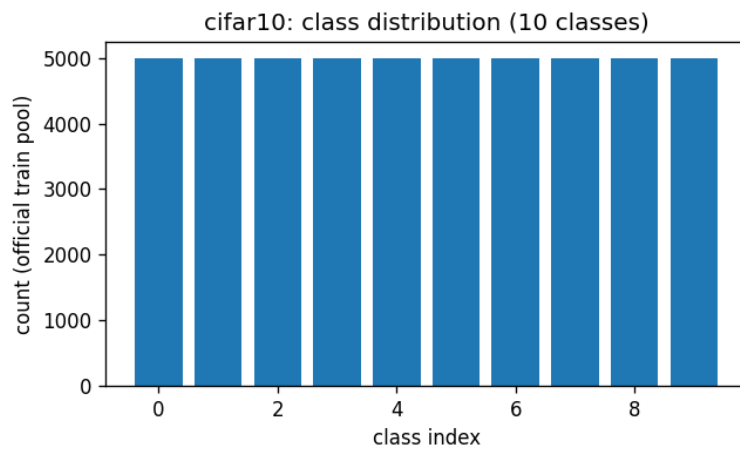


Figure 9: **cifar10** (dist). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar10_class_dist.png, produced by step1_datasets.py.

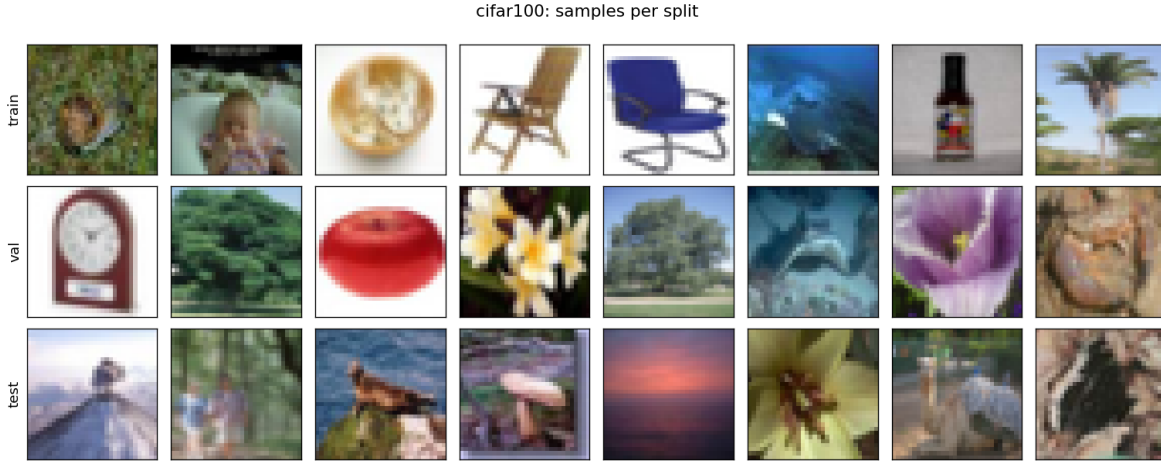


Figure 10: **cifar100** (samples). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar100_samples.png, produced by step1_datasets.py.

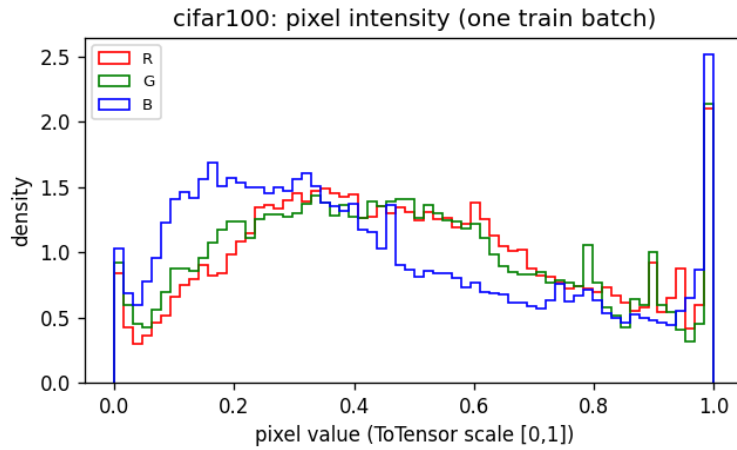


Figure 11: **cifar100** (hist). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar100_pixel_hist.png, produced by step1_datasets.py.

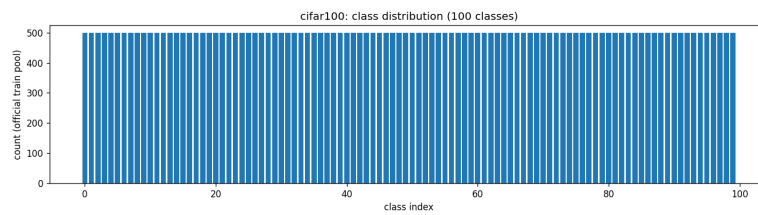


Figure 12: **cifar100** (dist). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/cifar100_class_dist.png, produced by step1_datasets.py.

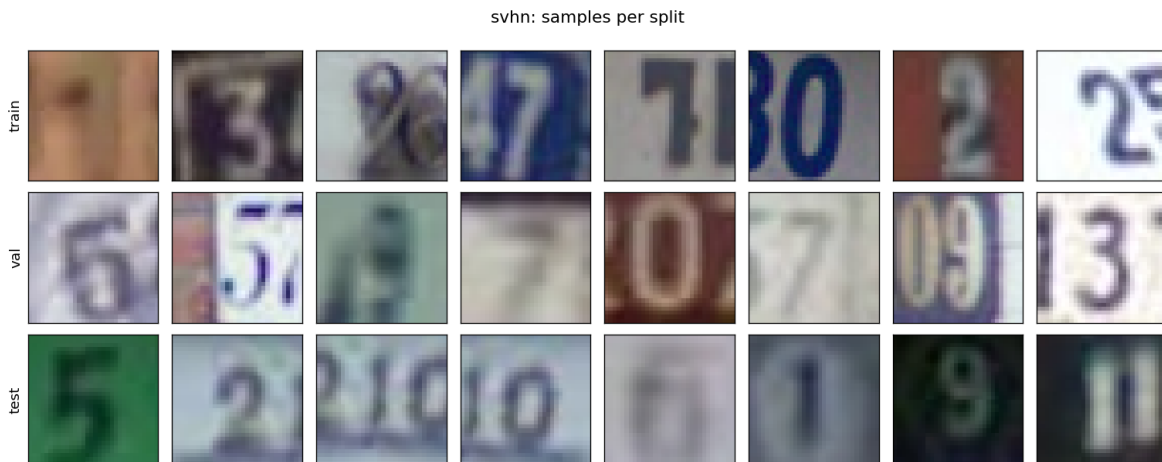


Figure 13: **svhn** (samples). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/svhn_samples.png, produced by step1_datasets.py.

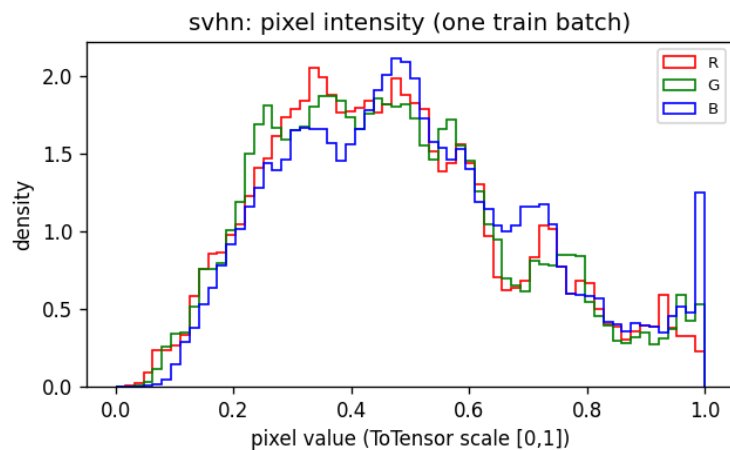


Figure 14: **svhn** (hist). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/svhn_pixel_hist.png, produced by step1_datasets.py.

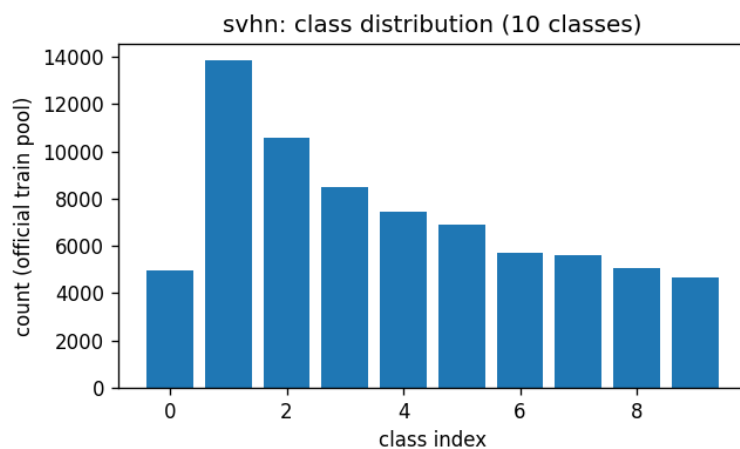


Figure 15: **svhn** (dist). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/svhn_class_dist.png, produced by step1_datasets.py.

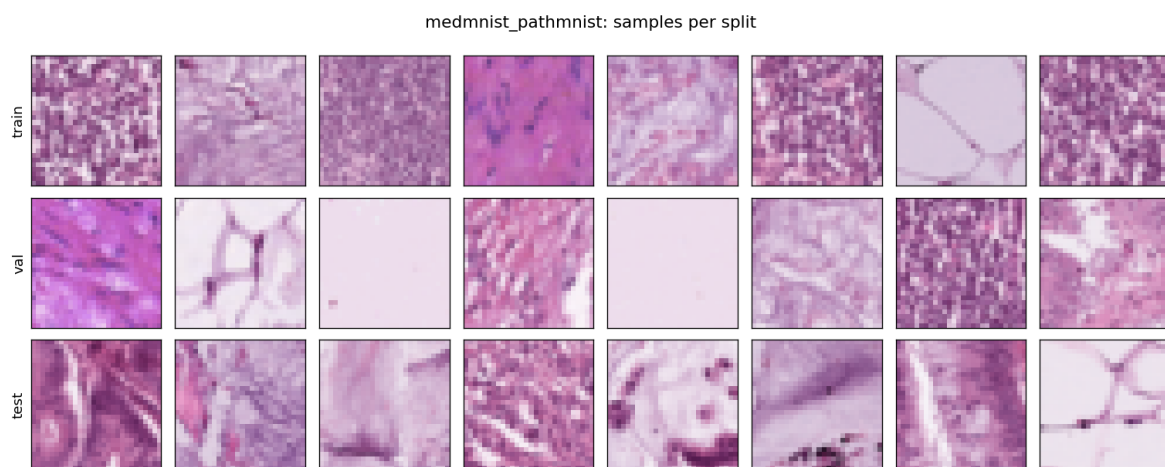


Figure 16: **medmnist_pathmnist** (samples). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pathmnist_samples.png, produced by step1_datasets.py.

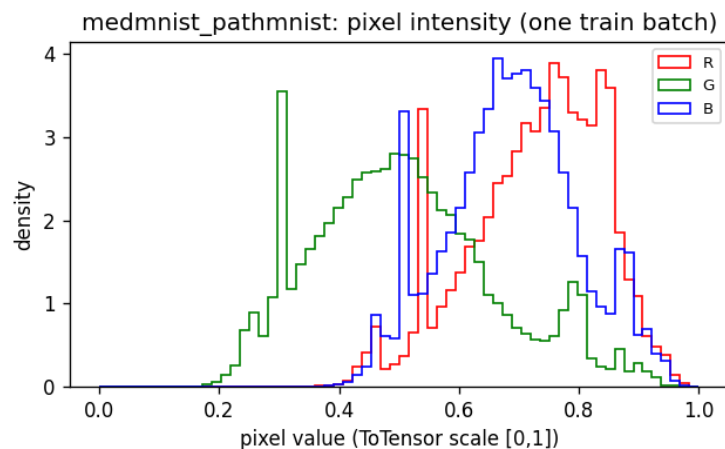


Figure 17: **medmnist_pathmnist** (hist). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pathmnist_pixel_hist.png, produced by step1_datasets.py.

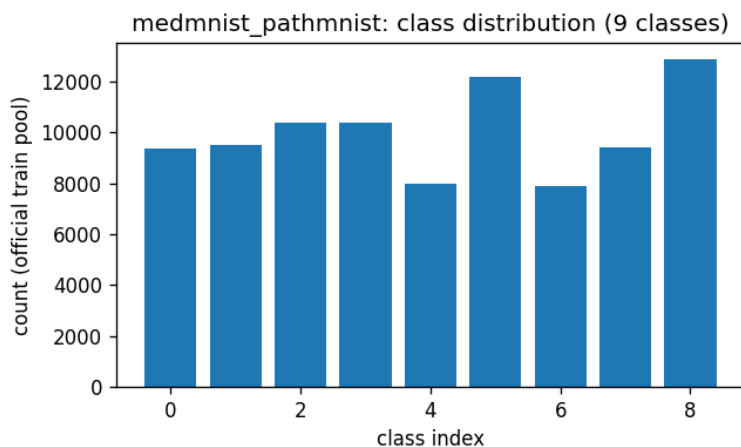


Figure 18: **medmnist_pathmnist** (dist). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pathmnist_class_dist.png, produced by step1_datasets.py.

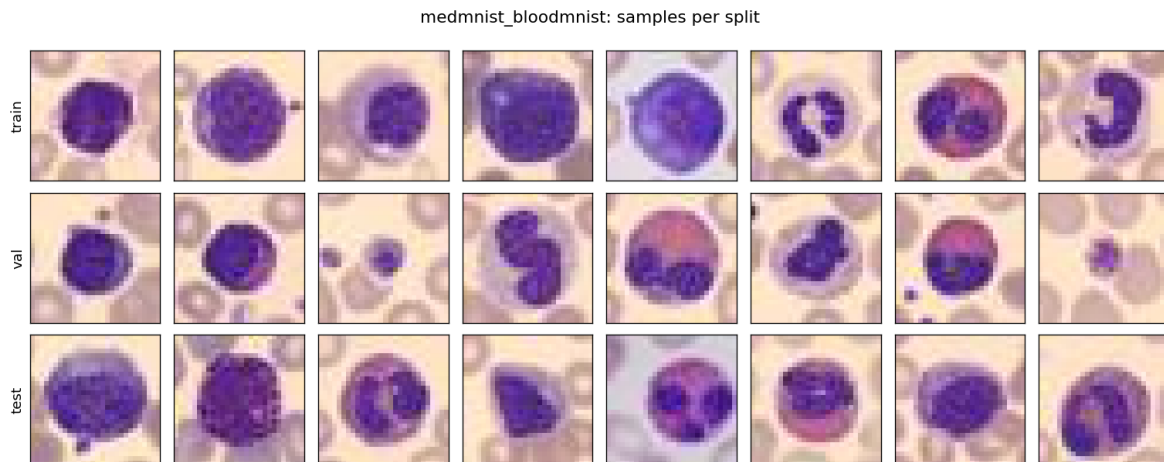


Figure 19: **medmnist_bloodmnist** (samples). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_bloodmnist_samples.png, produced by step1_datasets.py.

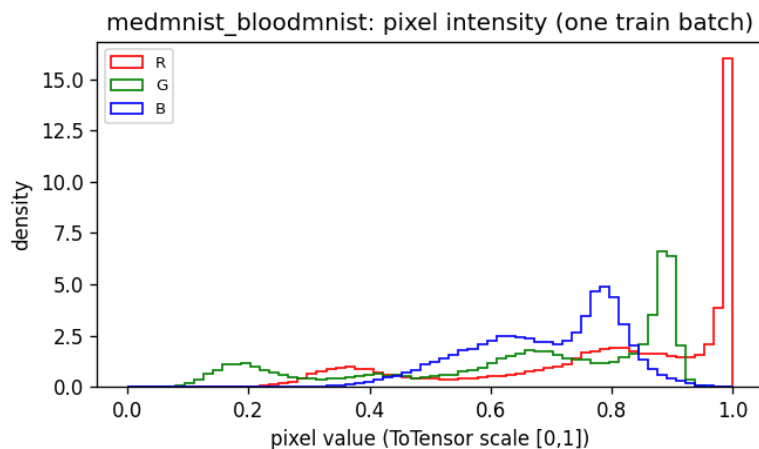


Figure 20: **medmnist_bloodmnist** (hist). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_bloodmnist_pixel_hist.png, produced by step1_datasets.py.

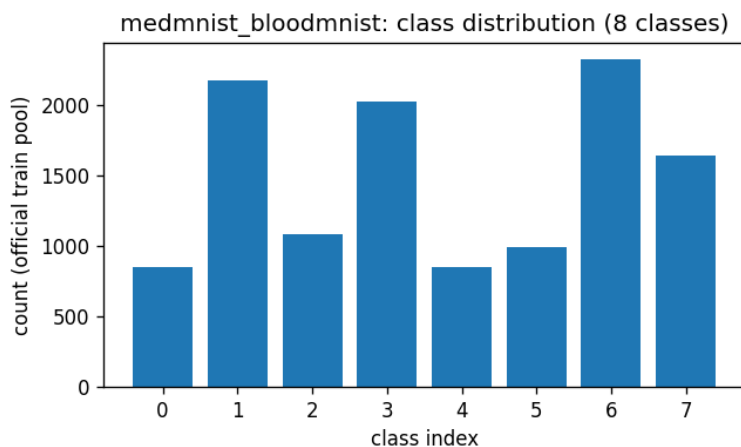


Figure 21: **medmnist_bloodmnist** (dist). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_bloodmnist_class_dist.png, produced by step1_datasets.py.

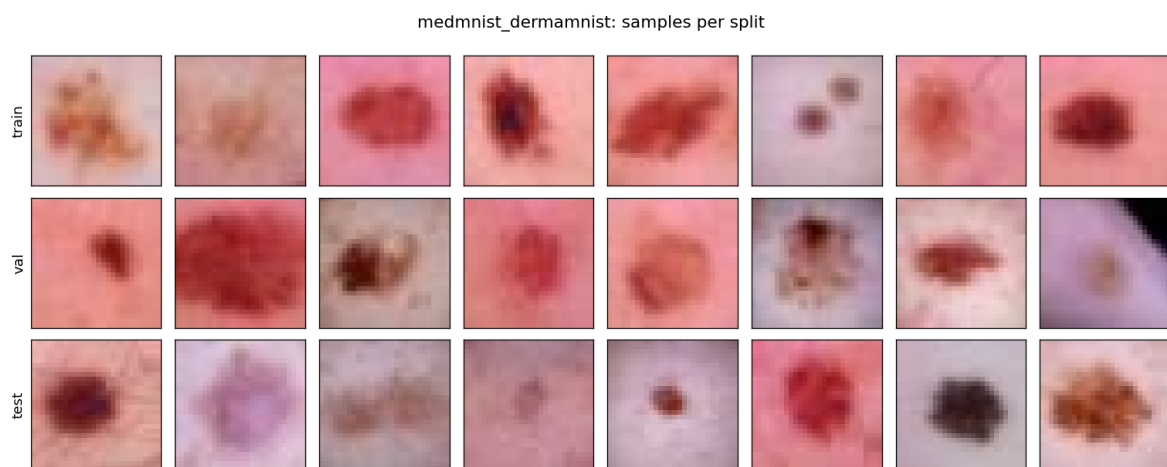


Figure 22: **medmnist_dermamnist** (samples). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_dermamnist_samples.png, produced by step1_datasets.py.

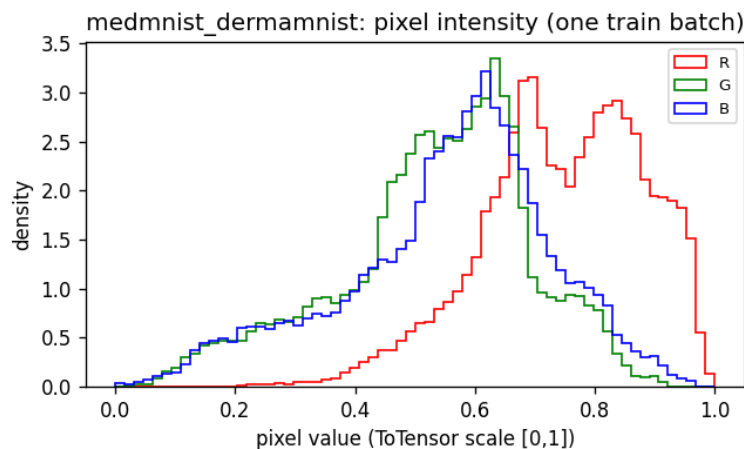


Figure 23: **medmnist_dermamnist** (hist). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_dermamnist_pixel_hist.png, produced by step1_datasets.py.

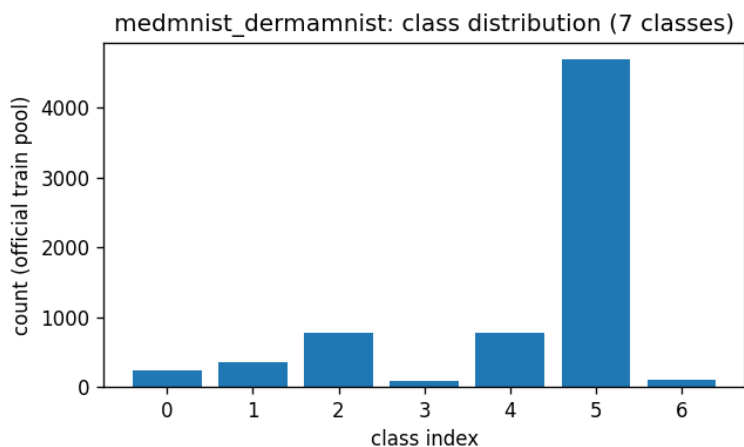


Figure 24: **medmnist_dermamnist** (dist). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_dermamnist_class_dist.png, produced by step1_datasets.py.

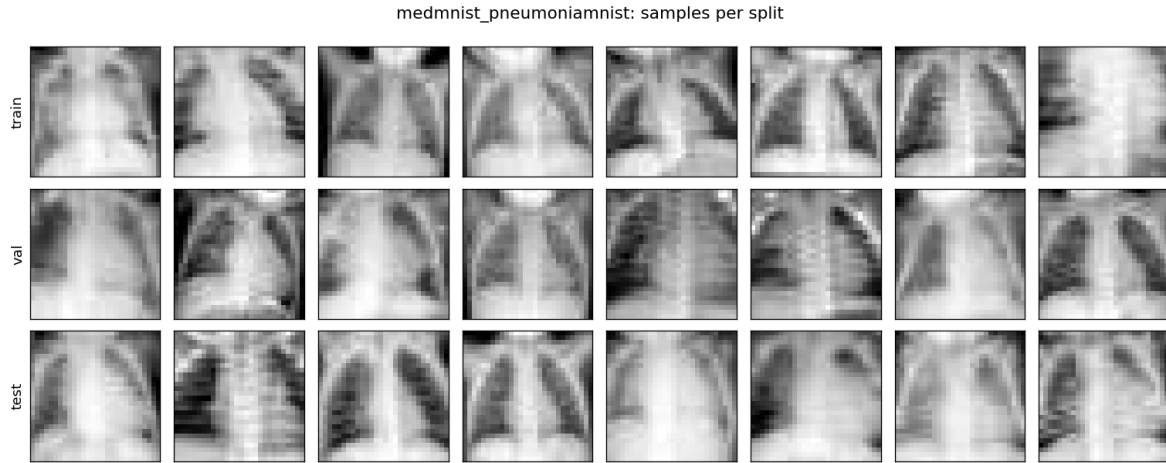


Figure 25: **medmnist_pneumoniamnist** (samples). Source: `medmnist.PneumoniaMNIST` v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pneumoniamnist_samples.png`, produced by `step1_datasets.py`.

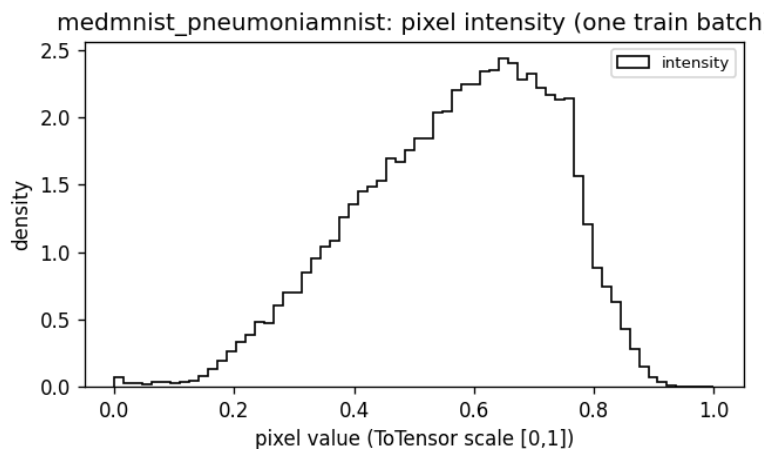


Figure 26: **medmnist_pneumoniarnist** (hist). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pneumoniarnist_pixel_hist.png, produced by step1_datasets.py.

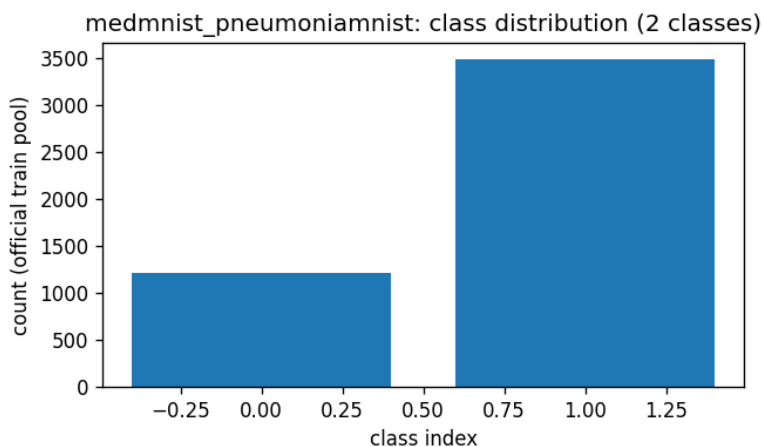


Figure 27: **medmnist_pneumoniarnist** (dist). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_pneumoniarnist_class_dist.png, produced by step1_datasets.py.

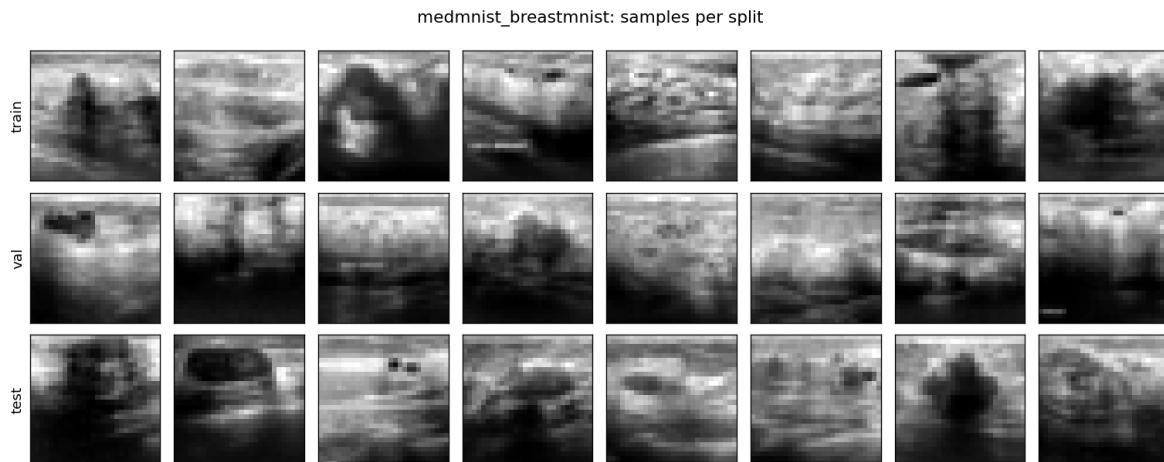


Figure 28: **medmnist_breastmnist** (samples). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_breastmnist_samples.png, produced by step1_datasets.py.

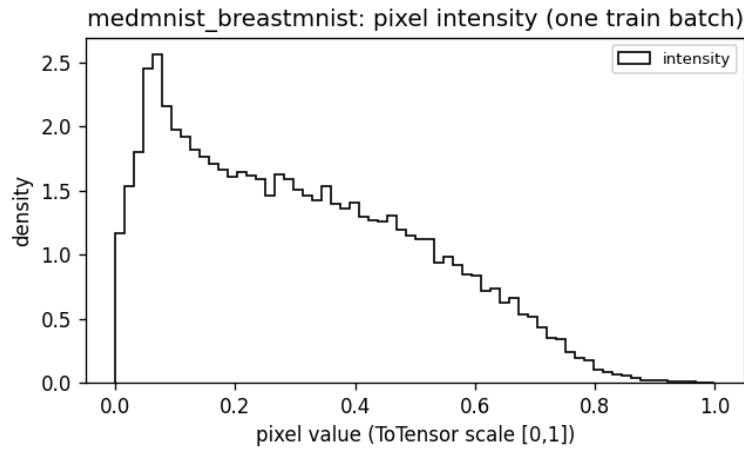


Figure 29: **medmnist_breastmnist** (hist). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_breastmnist_pixel_hist.png, produced by step1_datasets.py.

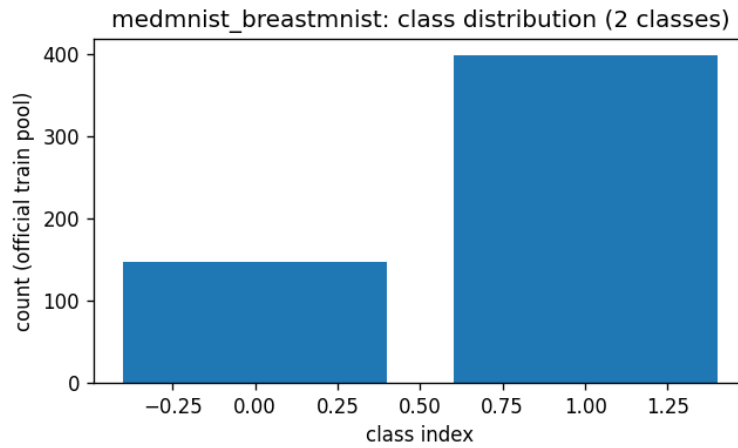


Figure 30: **medmnist_breastmnist** (dist). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_breastmnist_class_dist.png, produced by step1_datasets.py.

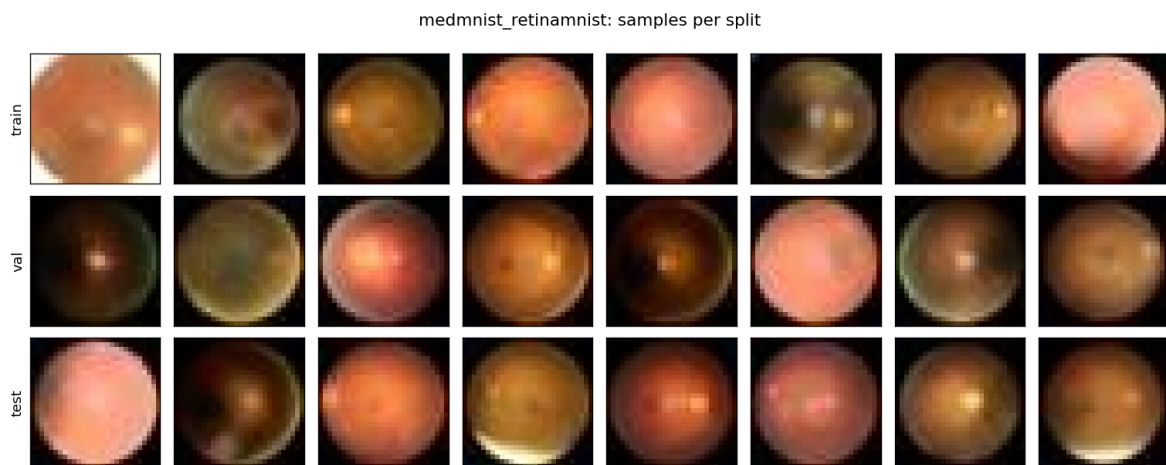


Figure 31: **medmnist_retinamnist** (samples). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_retinamnist_samples.png, produced by step1_datasets.py.

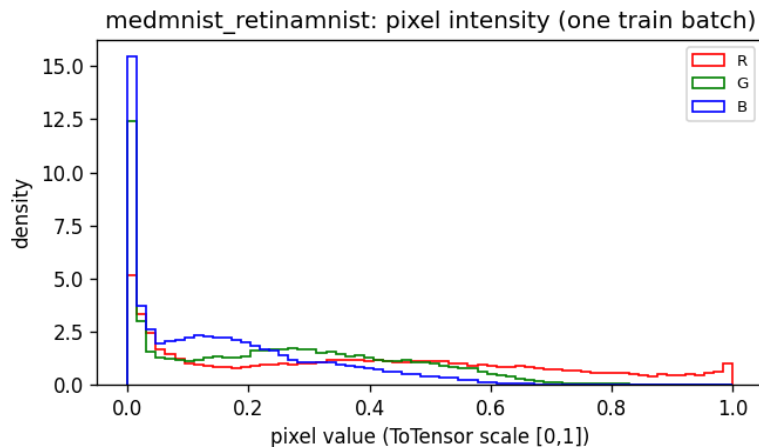


Figure 32: **medmnist_retinamnist** (hist). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_retinamnist_pixel_hist.png, produced by step1_datasets.py.

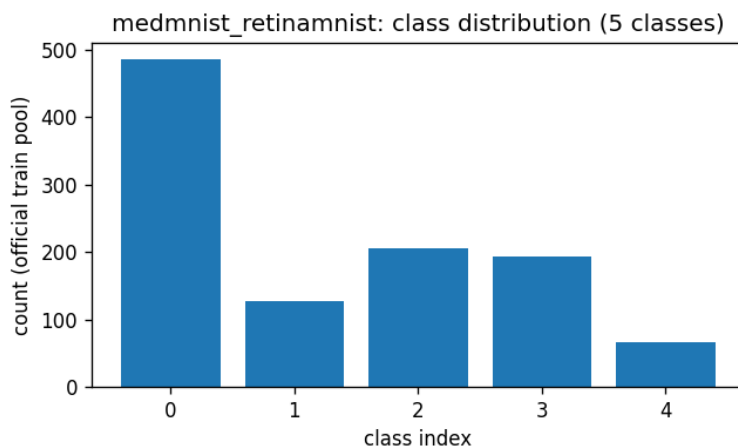


Figure 33: **medmnist_retinamnist** (dist). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_retinamnist_class_dist.png, produced by step1_datasets.py.

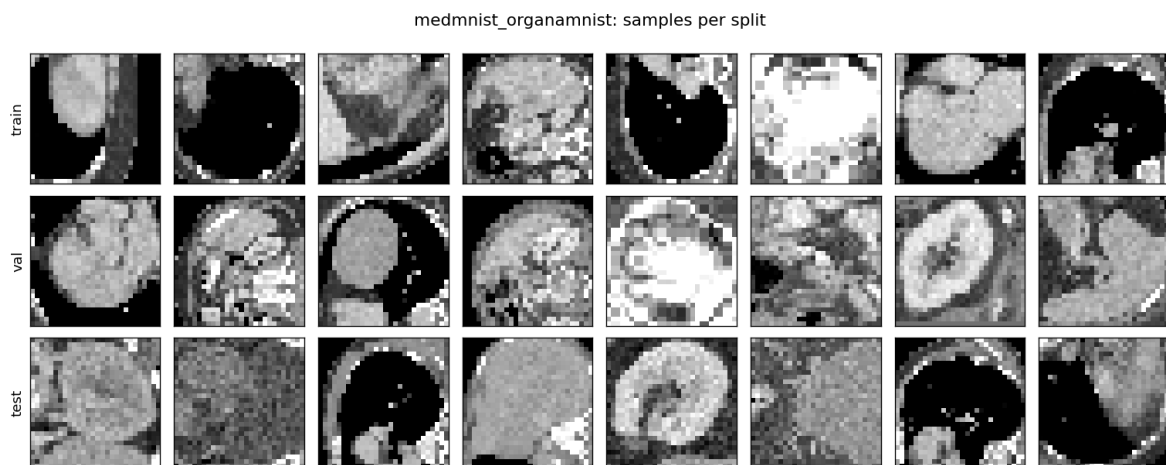


Figure 34: **medmnist_organamnist** (samples). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_organamnist_samples.png, produced by step1_datasets.py.

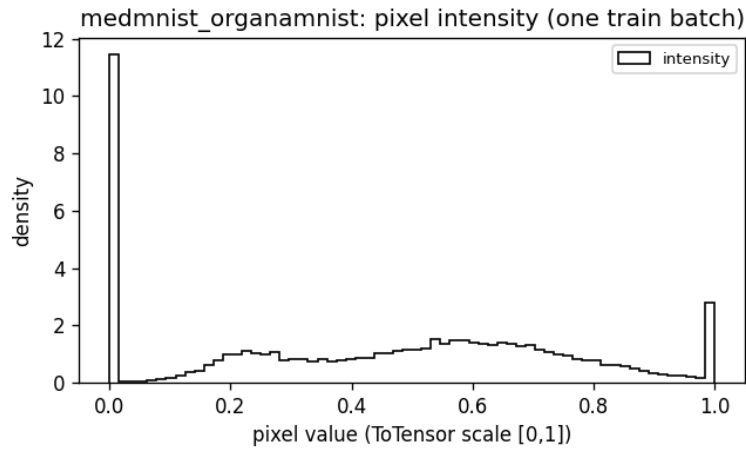


Figure 35: **medmnist_organamnist** (hist). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_organamnist-pixel_hist.png, produced by step1_datasets.py.

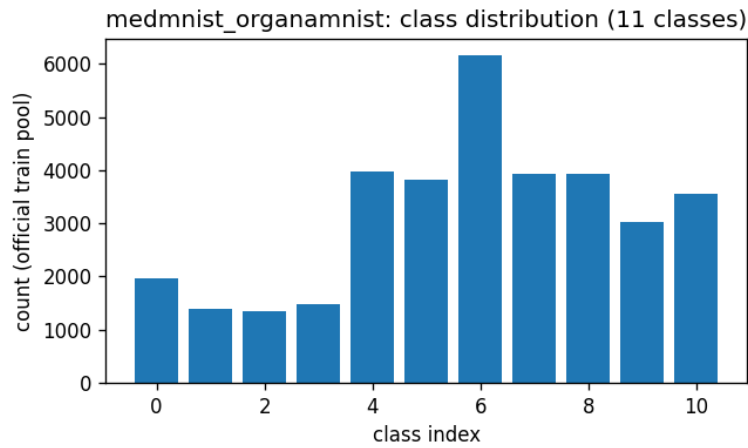


Figure 36: **medmnist_organamnist** (dist). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: outputs/step1_datasets/figures/medmnist_organamnist-class_dist.png, produced by step1_datasets.py.

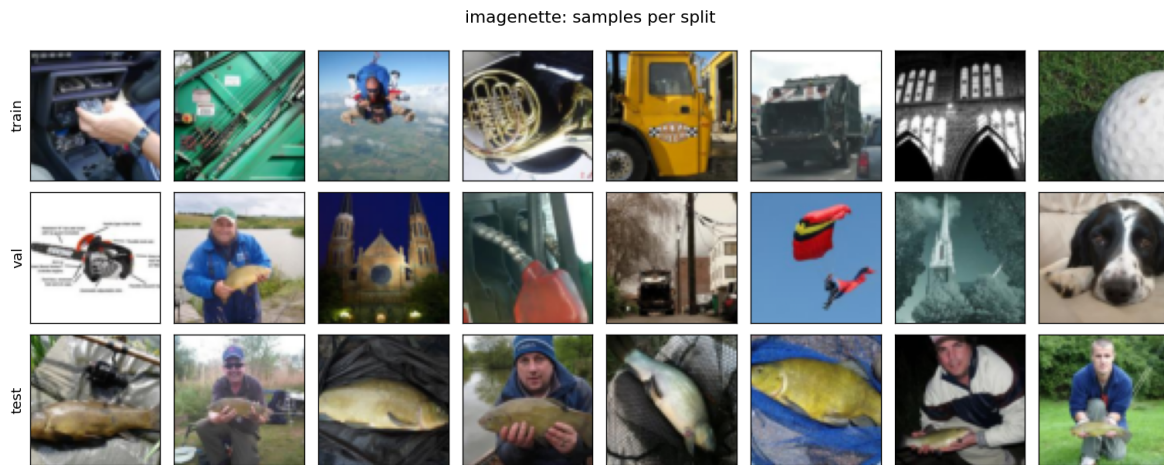


Figure 37: **imagenette** (samples). Source: `torchvision.datasets.Imagenette` 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/imagenette_samples.png`, produced by `step1_datasets.py`.

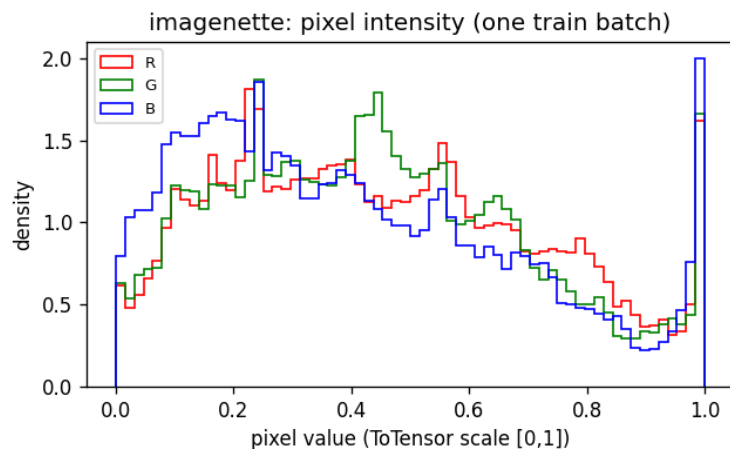


Figure 38: **imagenette** (hist). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/imagenette_pixel_hist.png, produced by step1_datasets.py.

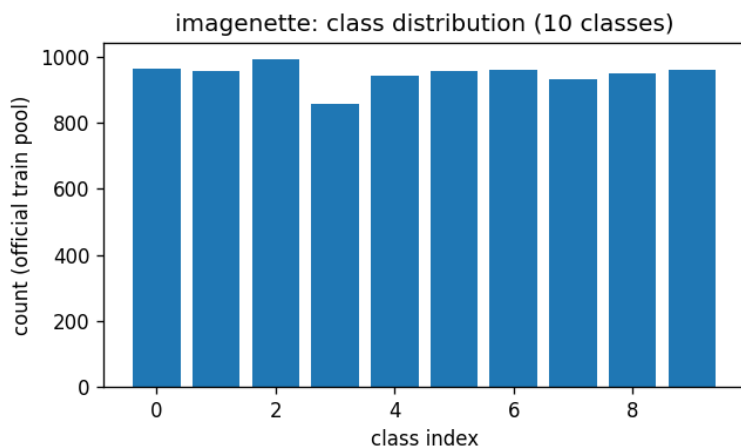


Figure 39: **imagenette** (dist). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/imagenette_class_dist.png, produced by step1_datasets.py.



Figure 40: **afhq** (samples). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_samples.png`, produced by `step1_datasets.py`.

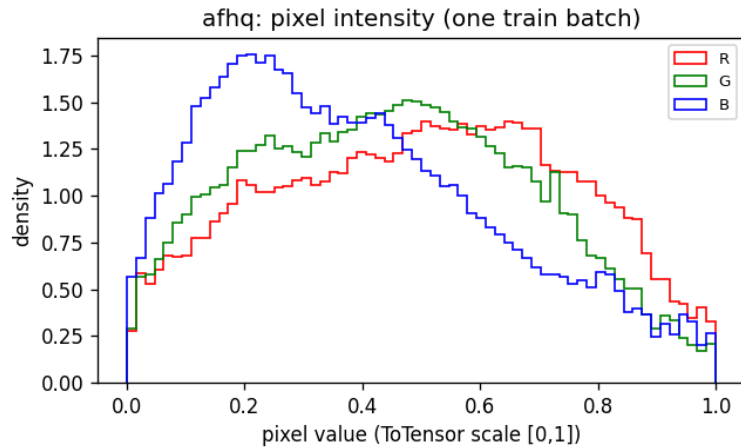


Figure 41: **afhq** (hist). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_pixel_hist.png`, produced by `step1_datasets.py`.

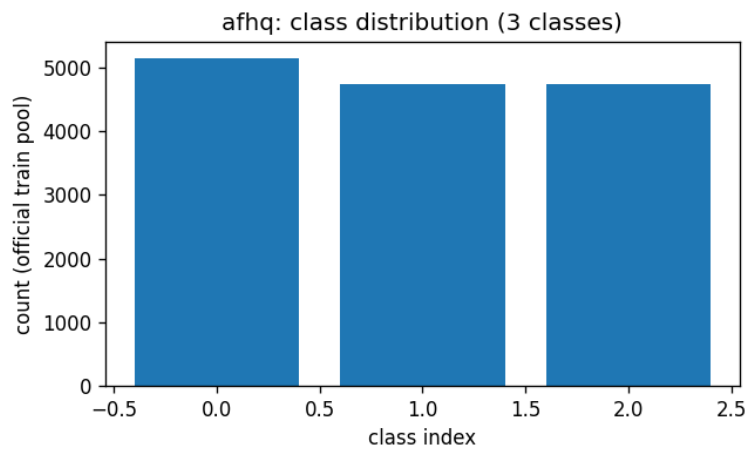


Figure 42: **afhq** (dist). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: outputs/step1_datasets/figures/afhq_class_dist.png, produced by step1_datasets.py.

2.6 Analysis

- Coverage: the available benchmarks span 28×28 grayscale (MNIST, several MedMNIST members) through 64×64 RGB natural images and faces (CIFAR, Imagenette, AFHQ, CelebA), which is the standard ladder for likelihood-based generative models: density results in bits/dim on MNIST/CIFAR-10, then perceptual-quality results on faces/animals at higher resolution.
- The pixel-intensity histograms show the strongly peaked, saturated distributions typical of 8-bit images (large masses at 0 and 1, e.g. MNIST background, CelebA skin tones). For normalizing flows this directly motivates uniform dequantization and a logit transform before the first coupling layer – to be implemented and ablated in the model step, not in the data step.
- Class-distribution figures document which datasets are usable for class-conditional modeling and how imbalanced they are; several MedMNIST members are markedly imbalanced, which matters for conditional likelihood evaluation.
- Per-channel means/stds recorded in each dataset’s JSON give the constants for any input normalization later steps choose to apply.
- Datasets marked unavailable (typically ImageNet, FFHQ, CelebA-HQ, LSUN, and CelebA when the Google Drive quota blocks download) are access-restricted or manually hosted; the pipeline documents the exact acquisition path and picks them up automatically once placed under `/data/image_benchmarks`.

3 Step 2: TarFlow – Transformer Autoregressive Flow

3.1 Methods

- Model: TarFlow from *Normalizing Flows are Capable Generative Models* (Zhai et al., Apple, arXiv:2412.06329). A TarFlow is a stack of T causal-ViT “metablocks”, each an autoregressive NVP flow over image patches; the patch order is flipped between consecutive blocks. Training maximizes exact likelihood; sampling inverts the blocks patch-by-patch with KV caching.
- Code: the *official* implementation (github.com/apple/ml-tarflow, Apple sample-code license) is used unmodified: `transformer_flow.py` is imported directly from a clone at `/dev_ws/ml-tarflow` (commit hash in `outputs/step2_tarflow/data/provenance.json`). Our driver `step2_tarflow.py` reimplements the surrounding training loop faithfully to the official `train.py` / `train_local.ipynb` / `evaluate_bpd.py`: AdamW ($\beta = (0.9, 0.95)$, wd 10^{-4}), cosine LR with one warmup epoch, inputs scaled to $[-1, 1]$, gaussian-noise runs in bf16 autocast, uniform-dequantization (likelihood) runs in fp32, and the official bits/dim formula (Gaussian prior term with the log 128 scaling correction, generalized from 3 channels to C).
- Data: datasets and deterministic splits come from step 1 (`step1_datasets.build_dataset`); an adapter rescales to $[-1, 1]$ and applies the official random horizontal flip where configured.
- Config: `configs/step2_tarflow.yml`. Outputs: `outputs/step2_tarflow/{data,figures}/`. Commands executed: `python step2_tarflow.py --runs mnist_toy` (GPU 0) and `python step2_tarflow.py --runs cifar10_uniform` (GPU 1), then `python step2_tarflow.py --collect`.

- Run **mnist_toy**: exact replication of Apple’s released MNIST toy experiment (`train_local.ipynb` defaults): class-conditional TarFlow [4-128-4-4] (patch-channels-blocks-layers), gaussian noise $\sigma = 0.1$, batch 256, lr 2×10^{-3} , 100 epochs. Deliverables: conditional sample grid (one row per digit) and the notebook’s Bayes-rule classifier evaluation $\arg \min_y \frac{1}{2} \mathbb{E}[z_y^2] - \log |\det|_y$ on the step-1 test split.
- Run **cifar10_uniform**: unconditional CIFAR-10 likelihood run in the paper’s likelihood configuration style: uniform dequantization, fp32, patch size 2 (as in their ImageNet 64 2.99-bpd config), TarFlow [2-256-8-4] ($\sim 26\text{M}$ params), batch 128, lr 2×10^{-4} , 60 epochs on one RTX 4090. Validation bpd is tracked every 5 epochs; final test bpd uses the official formula on the step-1 test split.
- Scale caveat (stated up front): the paper’s quantitative likelihood benchmark is *unconditional ImageNet 64×64* , where TarFlow [2-768-8-8] ($\sim 470\text{M}$ params, $8 \times \text{A100}$, 200 epochs) reaches **2.99 bpd**; the paper reports no MNIST or CIFAR-10 numbers. ImageNet is not yet staged (step 1), so this step validates the framework end-to-end at small scale and positions results against the classic CIFAR-10 flow literature (RealNVP 3.49, Glow 3.35, Flow++ 3.08 bpd).
- Sampling uses the plain reverse pass from Gaussian noise; the paper’s score-based denoising step (Tweedie correction) is not applied here.

3.2 Configuration (verbatim)

```
# Config for step2_tarflow.py -- TarFlow training/testing.
#
# tarflow_repo: clone of the OFFICIAL github.com/apple/ml-tarflow (model code
# imported unmodified; commit recorded in outputs provenance).

# GPU POLICY (2026-09-01): GPU0 ONLY. GPU1 failed 2026-08-31 and, though it
# enumerates again after the reboot, we do not use it. The recon-dev container
# is pinned to GPU0 (compose device_ids + CUDA_VISIBLE_DEVICES=0), so cuda:0 is
# the only valid device here; runs are sequential rather than one-per-GPU.
seed: 20260831
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step2_tarflow

runs:
# Exact replication of Apple’s released MNIST toy experiment
# (train_local.ipynb defaults): class-conditional TarFlow [4-128-4-4],
# gaussian noise 0.1, bs 256, lr 2e-3, 100 epochs.
mnist_toy:
  device: cuda:0
  dataset: mnist
  conditional: true
  img_size: 28
  channel_size: 1
  hflip: false
  patch_size: 4
  channels: 128
  blocks: 4
  layers_per_block: 4
  noise_type: gaussian
  noise_std: 0.1
```

```

cfg: 0
batch_size: 256
lr: 2.0e-3
epochs: 100
sample_freq: 10
sample_rows: 10      # one row per digit class, like the notebook's grid
sample_nrow: 10
eval_bpd: false      # gaussian-noise model: bpd not meaningful
eval_freq: 0
classifier_eval: true # Bayes-rule  $p(y|x)$  eval from the notebook

# Unconditional CIFAR-10 likelihood run in the paper's likelihood style:
# uniform dequantization, fp32 (official train.py disables amp for uniform),
# patch 2 (as in the ImageNet64 2.99-bpd config), scaled to 1x RTX 4090.
# Paper-scale reference: TarFlow [2-768-8-8] on unconditional ImageNet64,
# 8x A100, reaches 2.99 bpd; classic CIFAR-10 flows: RealNVP 3.49,
# Glow 3.35, Flow++ 3.08 bpd.
cifar10_uniform:
  device: cuda:0
  dataset: cifar10
  conditional: false
  img_size: 32
  channel_size: 3
  hflip: true          # official train.py uses RandomHorizontalFlip
  patch_size: 2
  channels: 256
  blocks: 8
  layers_per_block: 4
  noise_type: uniform
  cfg: 0
  batch_size: 128
  lr: 2.0e-4
  epochs: 60
  sample_freq: 10
  sample_rows: 8
  sample_nrow: 8
  eval_bpd: true
  eval_freq: 5
  classifier_eval: false

```

3.3 Results

run	dataset	model [p-c-b-l]	noise	params	epochs	result
mnist_toy	mnist	4-128-4-4	gaussian(0.1)	3.2M	100	Bayes acc 97.62%
cifar10_uniform	cifar10	2-256-8-4	uniform	25.9M	60	interrupted at epoch 9/60, val bp

Model column is [patch-channels-blocks-layers/block]. Full per-epoch metrics are in `outputs/step2_tarflow/data`

3.4 Results: figures

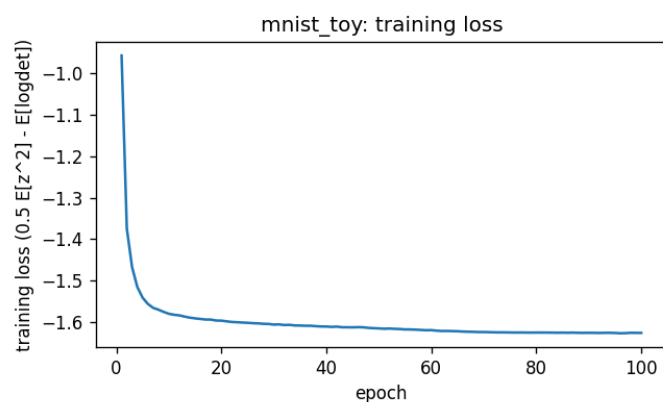


Figure 43: **mnist_toy** (loss). File: outputs/step2_tarflow/figures/mnist_toy_loss.png, produced by step2_tarflow.py.

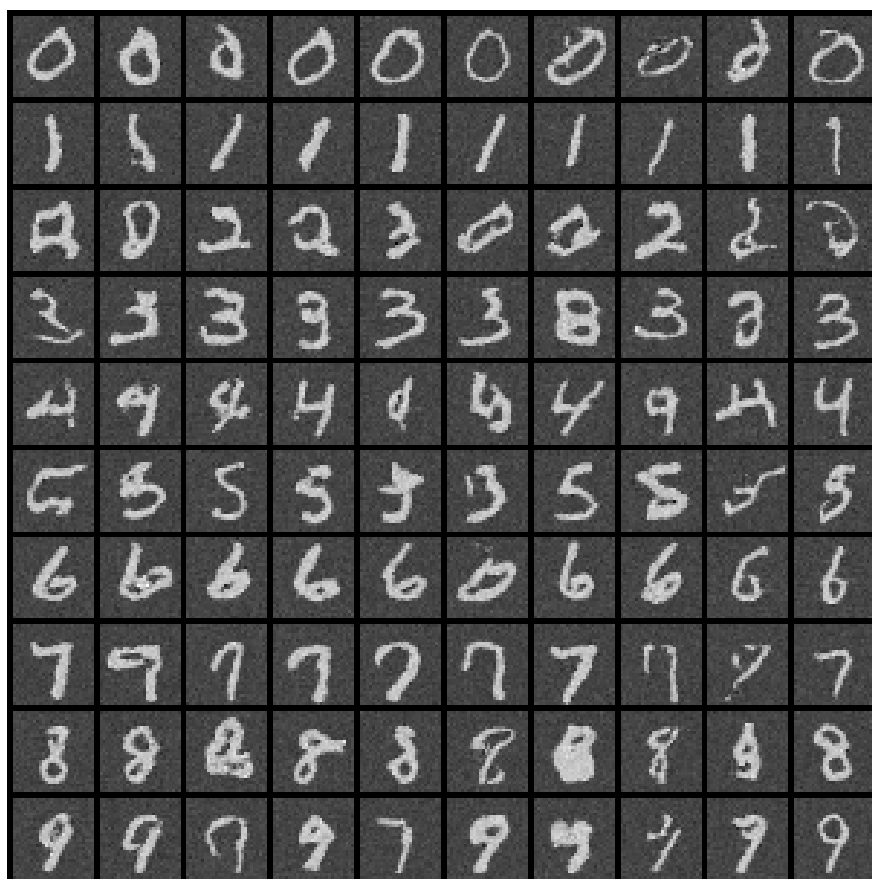


Figure 44: **mnist_toy**: samples at 010 epochs (reverse pass from Gaussian noise, no denoising step). File: outputs/step2_tarflow/figures/mnist_toy_samples_epoch010.png.

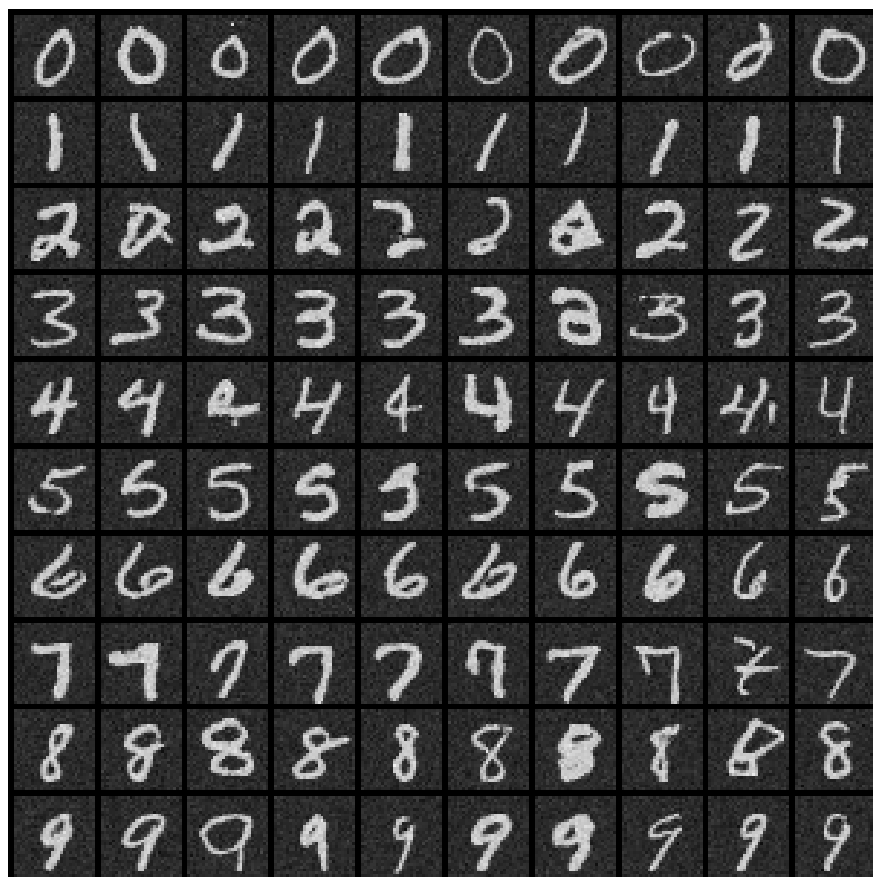


Figure 45: **mnist_toy**: samples at 100 epochs (reverse pass from Gaussian noise, no denoising step). File: `outputs/step2_tarflow/figures/mnist_toy_samples_epoch100.png`.

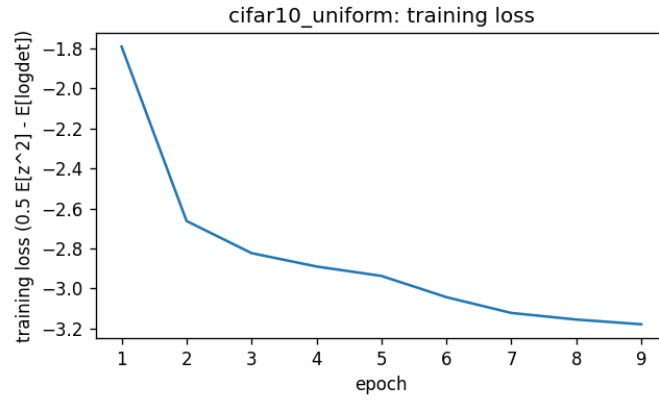


Figure 46: **cifar10_uniform** (loss). File: outputs/step2_tarflow/figures/cifar10_uniform_loss.png, produced by step2_tarflow.py.

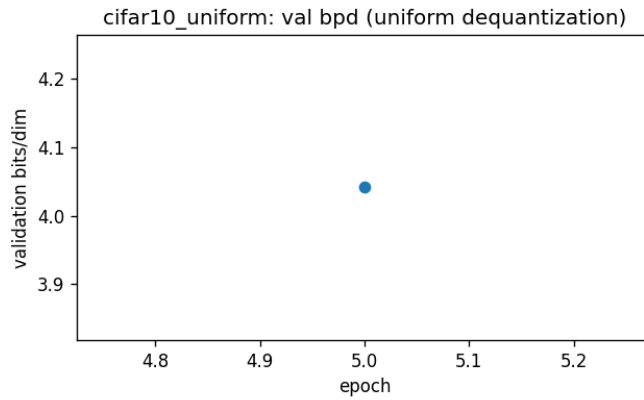


Figure 47: **cifar10_uniform** (bpd). File: outputs/step2_tarflow/figures/cifar10_uniform_bpd.png, produced by step2_tarflow.py.

3.5 Analysis

- The `mnist_toy` run reproduces Apple’s released toy example: training loss decreases smoothly from the first epochs (-0.70 at epoch 1 to ≈ -1.62 at epoch 100), the conditional sample grid shows clean, clearly class-consistent digits (one row per class), and the Bayes-rule classifier reaches **97.62%** test accuracy – confirming the trained TarFlow is a working class-conditional density model, which is exactly the property the official notebook demonstrates. Residual background grain in the samples is the $\sigma = 0.1$ training noise; the paper’s denoising step (not applied here) is designed to remove it.
- The `cifar10_uniform` run was *interrupted at epoch 9/60*: GPU 1 of the host failed under sustained load (fell off the PCIe bus, wedging the NVIDIA driver for new processes host-wide) and the run is queued to restart after a host reboot; the trainer now checkpoints optimizer state every epoch so future interruptions resume exactly. The partial trajectory is healthy and archived (metrics CSV + curves): train loss $-1.79 \rightarrow -3.18$ over 9 epochs and validation bpd **4.041** at epoch 5, the only bpd evaluation before the failure. For calibration: uniform-dequantization models start near 8 bpd at random init, classic CIFAR-10 flows reach RealNVP 3.49 / Glow 3.35 / Flow++ 3.08 at convergence, and the paper’s ImageNet-64 state of the art is 2.99 bpd at $\sim 18\times$ more parameters and $\sim 27\times$ more GPU-epochs. Exact replication of the paper’s headline number additionally requires staging ImageNet 64×64 (step-1 instructions).
- Framework verdict: the official TarFlow code trains stably out of the box under both noise regimes (bf16 gaussian, fp32 uniform), the likelihood pipeline (uniform dequantization + official bpd formula) produces sensible, steadily improving numbers, and inversion (sampling) works. The one incident so far was hardware, not the model.

4 Step 3: Library of invertible modules with verification

4.1 Methods

- Code: `invlib.py` (the library) and `step3_invertible_modules.py` (the verification and benchmark harness). Config: `configs/step3_invertible_modules.yml`. Outputs: `outputs/step3_invertible`.
- The library has two families with two contracts. **SeqTransform** implements TarFlow’s permutation-slot signature `forward(x, dim, inverse)` and contains only ORTHOGONAL maps ($|\det| = 1$): identity, flip, random permutation, orthonormal Haar wavelet, Walsh–Hadamard, DCT-II, discrete Hartley (real Fourier), fixed random rotations, and two learnable orthogonal parameterizations (products of Householder reflections; Cayley transform of a skew matrix), plus composition. Orthogonality is the exact condition under which the TarFlow metablock’s likelihood accounting (no logdet term for the reordering) and the invariance of the $\mathcal{N}(0, I)$ prior are preserved, so these are drop-in replacements: the causal transformer then autoregresses over transform *coefficients* – arbitrary orthogonal features of the patch sequence – instead of the raw patch order.
- **InvertibleModule** implements `forward(x)  $\rightarrow$  (y, logdet) / inverse(y)` with exact per-sample log-Jacobians: invertible diagonal (actnorm), Glow-style PLU dense linear and invertible 1×1 convolution (arXiv:1807.03039); invertible periodic (circular) convolutions decoupled in the frequency domain in 2D and 1D – the “invertible Fourier filter” – following Hoogetboom et al., arXiv:1901.11137; monotone elementwise cubic $x + ax^3$ ($a \geq 0$) with closed-form

Cardano inverse; general monotone odd polynomials (SOS-flow style, arXiv:1905.02325) inverted by bisection+Newton; invertible leaky ReLU; logit data transform; monotonic rational-quadratic splines (arXiv:1906.04032); Woodbury low-rank-plus-diagonal linear maps (Lu & Huang, NeurIPS 2020); affine coupling (RealNVP-style); and **SpectralFloorLinear**, the eigenvalue-floored low-rank map proposed for this project: $A = U \text{diag}(\lambda)U^\top + \lambda_0(I - UU^\top)$ with orthonormal $U \in \mathbb{R}^{D \times k}$ and $\lambda_i = \lambda_0 + \text{softplus}(\theta_i) \geq \lambda_0 > 0$ – a full-rank invertible stand-in for a rank- k eigendecomposition in which every unretained eigenvalue is set to the floor λ_0 (necessarily $\leq$ each retained one) instead of zero; the inverse and $\log \det = \sum_i \log \lambda_i + (D - k) \log \lambda_0$ are analytic.

- Verification per module (command: `python step3_invertible_modules.py`): (1) reconstruction $\max |x - f^{-1}(f(x))|$ in fp32 and fp64; (2) the module’s logdet against $\log |\det|$ of the full autograd Jacobian (fp64, one sample); (3) $\|Q^\top Q - I\|_\infty$ for the orthogonal family; (4) a TarFlow integration test: each SeqTransform is installed in an actual `MetaBlock` (perturbed from identity) and the sequential reverse pass must invert the forward pass; (5) cost: median forward/inverse wall time and parameter count/bytes.
- The harness caught two real implementation bugs before any training used the library (a U/V swap in the Woodbury inverse, exposed by identical fp32/fp64 reconstruction error; and a determinant-sign over-restriction: the Hartley matrix has $\det = -1$, which is valid for flows since only $\log |\det|$ enters the likelihood).

4.2 Configuration (verbatim)

```
# Config for step3_invertible_modules.py -- invertible-library verification.

seed: 20260901
output_root: outputs/step3_invertible_modules
tarflow_repo: /dev_ws/ml-tarflow # for the MetaBlock permutation-slot test

dims:
  flat: 64 # D for flat (B,D) modules
  image: [2, 8, 8] # (C,H,W) for image modules (small so the autograd
    # Jacobian logdet check stays cheap: D = C*H*W = 128)
  seq: 64 # T for sequence transforms (power of 2: Haar/Hadamard)

feature_domain: # context for the 2D feature-domain tokenizers
  img_size: 32
  patch_size: 4 # T = 64 tokens of 16 coefficients

batch: 64 # batch for reconstruction + timing
timing_reps: 25 # median over this many timed calls
viz_patch: 4 # patch size for the MNIST coefficient gallery (32/4 -> T=64)

tolerances:
  recon_fp32: 1.0e-4 # max |x - f^-1(f(x))| in fp32
  logdet: 1.0e-3 # |logdet_module - slogdet(autograd jacobian)| (fp64)
  orthogonality: 1.0e-5 # ||Q^T Q - I||_inf (fp64)
  metablock: 1.0e-3 # TarFlow MetaBlock forward->reverse roundtrip (fp32)
```


4.3 Results: verification

module	kind	params	recon fp32	recon fp64	logdet err	orth err	fwd/inv ms	status
seq_identity	seq	0	0.0e+00	0.0e+00	0.0e+00	0.0e+00	0.00/0.00	PASS
seq_flip	seq	0	0.0e+00	0.0e+00	0.0e+00	0.0e+00	0.01/0.00	PASS
seq_random_perm	seq	0	0.0e+00	0.0e+00	0.0e+00	0.0e+00	0.01/0.00	PASS
seq_haar	seq	0	4.8e-07	1.0e-07	7.2e-07	2.2e-08	0.02/0.01	PASS
seq_hadamard	seq	0	8.3e-07	0.0e+00	1.8e-15	0.0e+00	0.02/0.01	PASS
seq_dct	seq	0	1.2e-06	1.4e-07	2.6e-07	9.6e-09	0.02/0.01	PASS
seq_hartley	seq	0	9.5e-07	1.6e-07	2.5e-07	4.6e-08	0.02/0.01	PASS
seq_rand_ortho	seq	0	1.3e-06	1.6e-07	2.6e-08	3.2e-08	0.02/0.01	PASS
seq_householder	seq	512	3.6e-06	6.7e-15	2.1e-15	1.8e-07	0.14/0.14	PASS
seq_cayley	seq	4096	2.6e-06	6.7e-15	4.2e-16	5.1e-07	0.07/0.07	PASS
invertible_diagonal	flat	128	2.4e-07	4.4e-16	1.1e-16	–	0.01/0.01	PASS
actnorm2d	image	4	4.8e-07	4.4e-16	2.0e-15	–	0.02/0.01	PASS
plu_linear	flat	8256	3.1e-06	9.1e-15	6.7e-16	–	0.06/0.10	PASS
invertible_1x1_conv	image	10	4.8e-07	8.9e-16	6.2e-15	–	0.07/0.08	PASS
circular_conv2d	image	18	8.3e-07	2.7e-15	3.0e-15	–	0.07/0.09	PASS
circular_conv1d_fourier_filter	flat	5	8.3e-07	1.3e-15	2.4e-07	–	0.23/0.20	PASS
spectral_floor_linear	flat	521	4.8e-07	8.9e-16	8.0e-15	–	0.06/0.05	PASS
woodbury_linear	flat	1088	4.8e-07	8.9e-16	0.0e+00	–	0.07/0.07	PASS
monotonic_cubic	flat	1	2.4e-07	5.3e-15	0.0e+00	–	0.03/0.11	PASS
monotonic_polynomial	flat	3	1.2e-07	1.1e-16	0.0e+00	–	0.06/2.96	PASS
invertible_leaky_relu	flat	0	0.0e+00	0.0e+00	6.9e-08	–	0.02/0.01	PASS
logit	flat	0	1.2e-07	2.2e-16	0.0e+00	–	0.05/0.01	PASS
rq_spline	flat	23	4.8e-07	4.4e-16	1.3e-15	–	0.28/0.25	PASS
affine_coupling	flat	6272	0.0e+00	0.0e+00	0.0e+00	–	0.07/0.07	PASS
feat_haar2d	feature	0	4.8e-07	2.4e-07	0.0e+00	1.2e-07	0.23/0.31	PASS
feat_dct2d	feature	0	9.5e-07	2.0e-07	0.0e+00	0.0e+00	0.23/0.30	PASS

sequence transform	MetaBlock roundtrip err	status
seq_identity	3.6e-07	PASS
seq_flip	2.4e-07	PASS
seq_random_perm	4.8e-07	PASS
seq_haar	4.8e-07	PASS
seq_hadamard	1.2e-06	PASS
seq_dct	1.2e-06	PASS
seq_hartley	1.3e-06	PASS
seq_rand_ortho	1.7e-06	PASS
seq_householder	1.9e-06	PASS
seq_cayley	2.9e-06	PASS
feat_haar2d	7.2e-07	PASS
feat_dct2d	1.2e-06	PASS

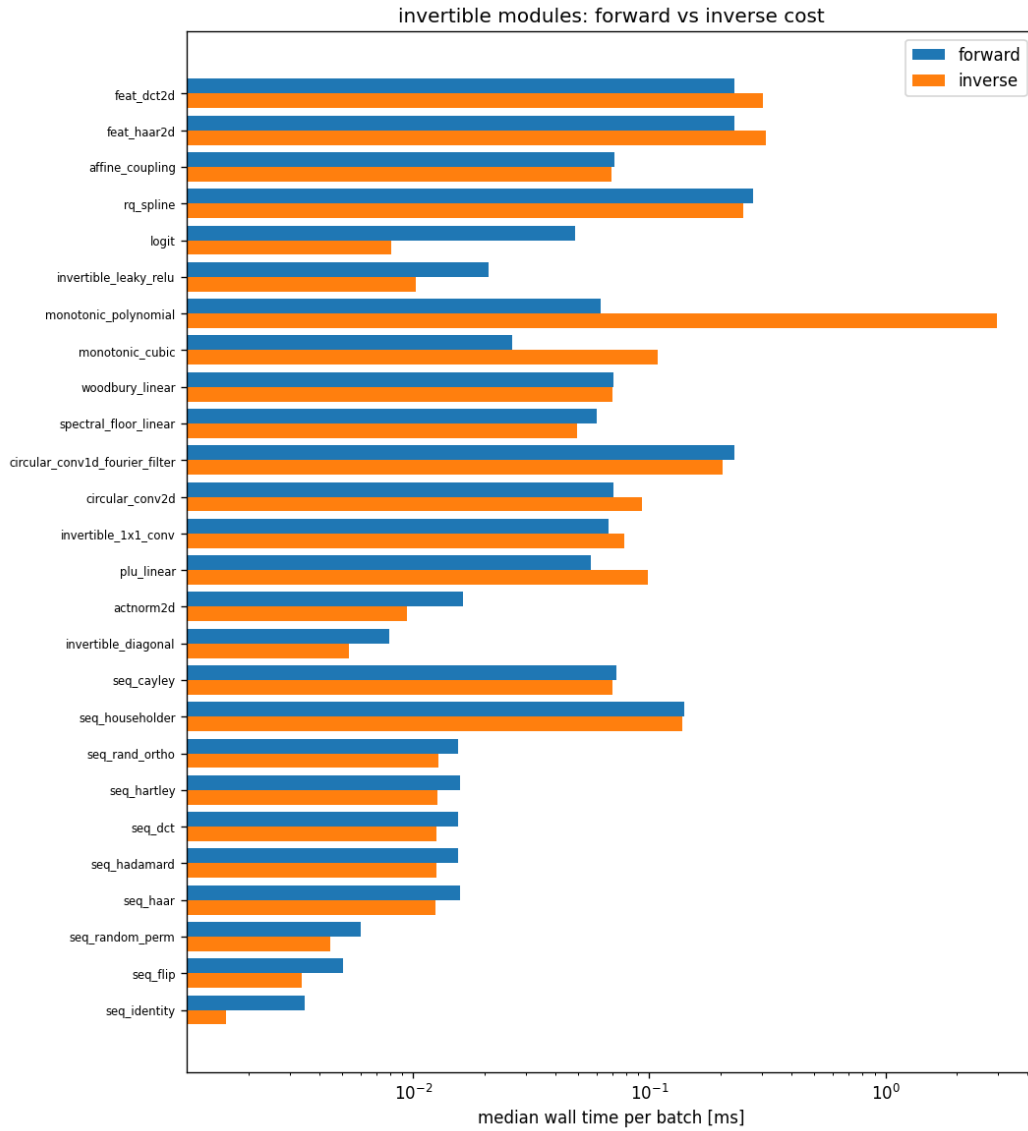


Figure 48: Median forward/inverse wall time per batch of 64 (CPU; the GPU rerun uses the same harness). File: `outputs/step3_invertible_modules/figures/timing.png`.

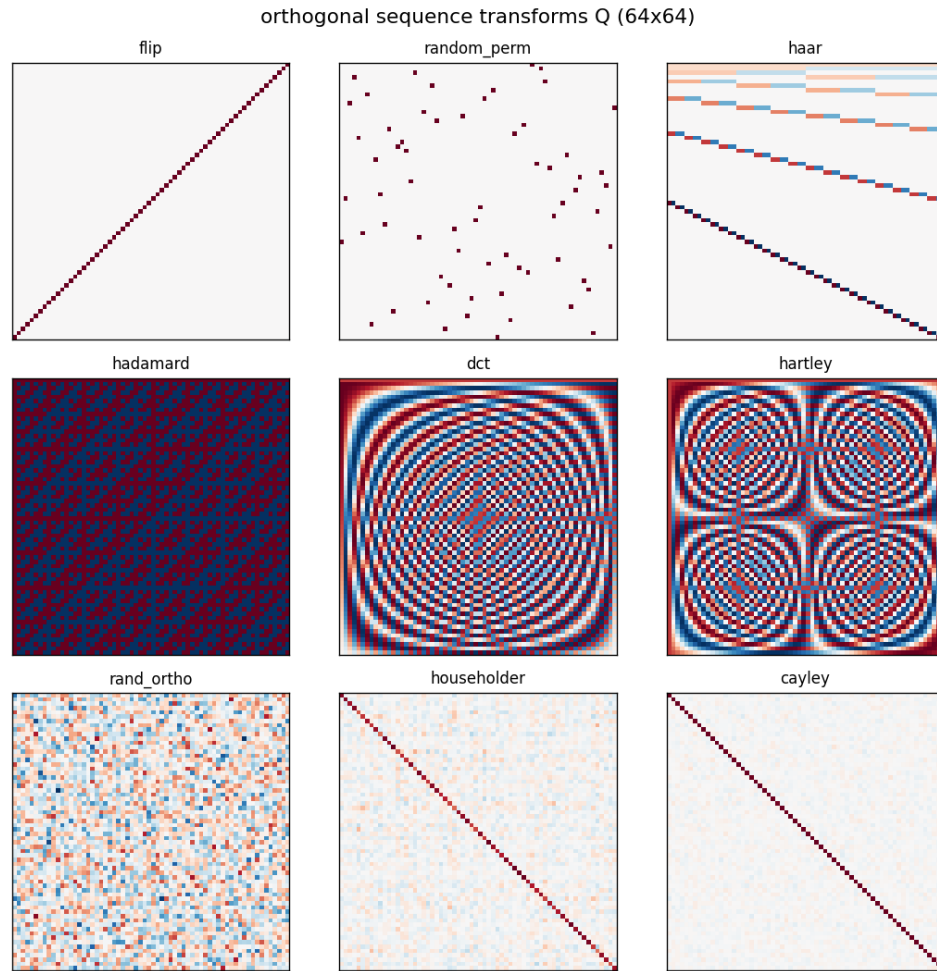


Figure 49: The orthogonal sequence transforms Q (64×64) available for the TarFlow permutation slot. File: `outputs/step3_invertible_modules/figures/orthogonal_matrices.png`.

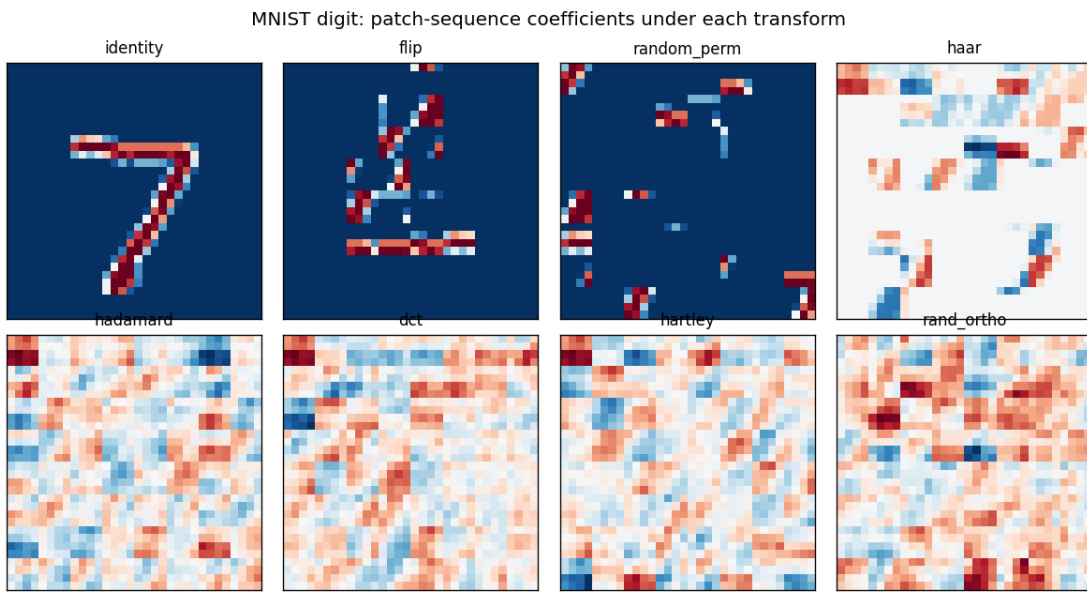


Figure 50: An MNIST digit’s patch sequence under each transform – the “features” the autoregressive model would model. Haar concentrates the digit into hierarchical detail coefficients; DCT/Hartley/Hadamard/random rotations spread it into global features. File: `outputs/step3_invertible_modules/figures/mnist_coefficients.png`.

4.4 Analysis

- All 26 modules PASS (24 original + the two 2D feature-domain tokenizers of step 5, verified here with roundtrip, norm preservation, DC-concentration and MetaBlock tests): fp32 reconstruction errors are at roundoff (10^{-7} – 10^{-6}), every analytic logdet matches the autograd Jacobian to 10^{-3} or (usually) far better in fp64, all orthogonal maps satisfy $\|Q^\top Q - I\|_\infty < 10^{-5}$, and every SeqTransform passes the end-to-end MetaBlock forward→reverse roundtrip at $\sim 10^{-6}$ – i.e. the whole orthogonal family is verified TarFlow-compatible before any training depends on it.
- Cost: fixed orthogonal transforms are parameter-free and as cheap as the baseline permutations at this scale (dense 64×64 matmul); learnable orthogonal maps (Householder, Cayley) cost more per call because Q is rebuilt each forward. Elementwise and FFT-based modules sit in between; inverse cost exceeds forward cost mainly for iterative inverses (monotone polynomial) and triangular solves (PLU).

5 Step 4: Ablation – what should TarFlow autoregress over?

5.1 Methods

- Code: `step4_tarflow_ablation.py`; config: `configs/step4_tarflow_ablation.yml`; outputs: `outputs/step4_tarflow_ablation/{data,figures}/`.
- Question: with everything else identical, which orthogonal feature basis in the permutation slot gives the best generative performance in a FIXED small number of epochs?
- Protocol: unconditional MNIST zero-padded to 32×32 (padding keeps 8-bit values valid for dequantization; $T = (32/4)^2 = 64$ is a power of two so Haar/Hadamard apply), TarFlow [4-128-4-2], uniform dequantization in fp32, identical seed (model init and data order) for every variant. Odd blocks compose the transform with a flip so consecutive blocks autoregress in opposite coefficient orders, mirroring the official Identity/Flip alternation.
- Variants (one SeqTransform per metablock): `baseline_flip` (official), `identity_only` (control: no reordering), `random_perm`, `haar`, `hadamard`, `dct`, `hartley`, `rand_ortho` (per-block seeds), `householder` (learnable, 16 reflections), `cayley` (learnable).
- Tracked per variant: val bits/dim per epoch, final test bits/dim, wall time, transform parameter overhead, and (on GPU) peak memory; plus a sample grid from the reverse pass.
- Budget profiles: **full** (GPU: 120 epochs, all 54k train) and **reduced** (CPU: 6 epochs, 12k subset). The run reported here is the **full** GPU profile: all ten variants at 120 epochs on one RTX 4090, identical init, data order and budget. An earlier version of this document reported the **reduced** CPU profile (6 epochs, 12k subset), run while the host GPUs were unavailable; §5.4 records where the two disagree, because they disagree substantially.

5.2 Configuration (verbatim)

```
# Config for step4_tarflow_ablation.py -- permutation-slot ablation.
#
# Every variant is trained with IDENTICAL model init, data order and budget;
# only the per-block sequence transform differs (all orthogonal, so the
```

```

# TarFlow likelihood accounting is untouched). Metric of merit: validation
# bits/dim after the fixed budget ("generative performance in finite epochs").

seed: 20260901
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step4_tarflow_ablation

dataset: mnist
img_size: 32          # zero-padded from 28 so  $T = (32/4)^2 = 64$  is a power of 2
channel_size: 1
patch_size: 4
noise_type: uniform # likelihood configuration (fp32, official bpd formula)
grad_clip: 1.0      # uniform stabilizer for ALL variants: coefficient bases
                   # (e.g. Haar, DC coeff  $\sim \sqrt{T}$  x mean) have much larger
                   # per-coordinate scales than raw patches and diverge
                   # without it at this lr (first no-clip run: haar -> nan)

model:              # small TarFlow so many variants fit in the budget
  channels: 128
  blocks: 4
  layers_per_block: 2

sample_nrow: 6      # 6x6 sample grid per variant

budget:
  full:             # GPU profile (auto-selected when CUDA is available)
    epochs: 120
    train_subset: 0      # 0 = all 54k
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
    sample: true
  reduced:          # CPU profile (GPU down 2026-09-01; rerun full after reboot)
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3
    sample: true

variants:
  - baseline_flip    # official TarFlow: Identity / Flip alternation
  - identity_only     # control: no reordering at all
  - random_perm       # fixed random patch permutation per block
  - haar              # orthonormal Haar wavelet over the patch sequence
  - hadamard          # Walsh-Hadamard
  - dct               # DCT-II (frequency features)
  - hartley           # discrete Hartley (real Fourier)
  - rand_ortho        # fixed random rotation (different seed per block)
  - householder       # learnable orthogonal (16 Householder reflections)
  - cayley            # learnable orthogonal (Cayley transform)

```

5.3 Results

rank	variant	val bpd	test bpd	xform params	min	mem MB
1	baseline_flip	1.1639	1.1709	0	9.4	598
2	cayley	1.2058	1.2027	16384	13.3	601
3	haar	1.6156	1.6079	0	10.1	598
4	random_perm	1.7499	2.1574	0	9.5	598
5	householder	2.2465	2.1992	4096	47.6	603
6	hadamard	2.7846	2.7795	0	10.0	598
7	dct	2.9531	2.9431	0	9.9	598
8	rand_ortho	3.1415	3.1220	0	9.8	598
9	hartley	3.1783	3.1647	0	9.8	598
10	identity_only	13.0945	11.1675	0	9.3	598

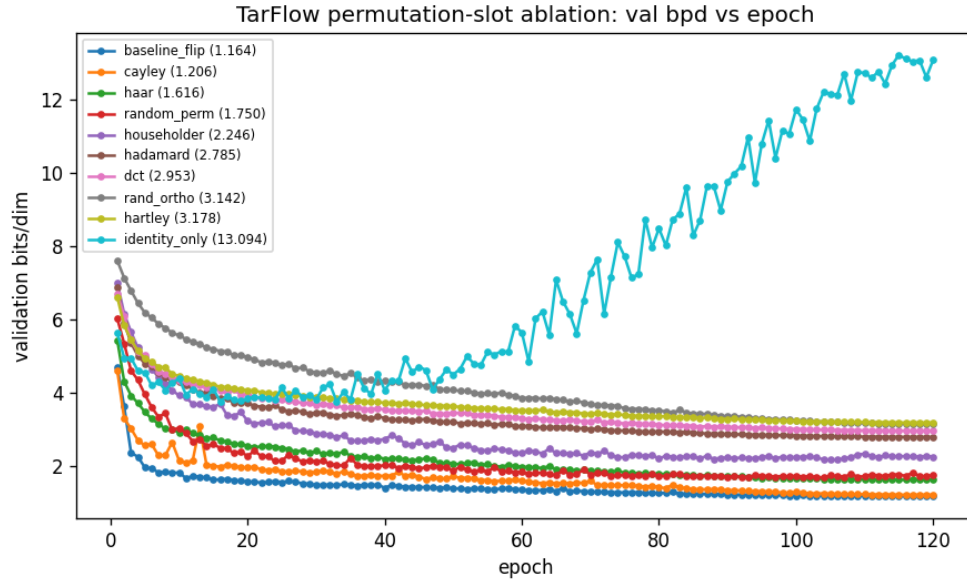


Figure 51: Validation bits/dim per epoch for every permutation-slot variant (legend sorted by final val bpd). File: outputs/step4_tarflow_ablation/figures/bpd_overlay.png.



Figure 52: **baseline_flip**: samples after the fixed budget (val bpd 1.1639). File: outputs/step4_tarflow_ablation/figures/baseline_flip_samples.png.



Figure 53: **cayley**: samples after the fixed budget (val bpd 1.2058). File: outputs/step4_tarflow_ablation/figures/cayley_samples.png.



Figure 54: **haar**: samples after the fixed budget (val bpd 1.6156). File: `outputs/step4_tarflow_ablation/figures/haar_samples.png`.



Figure 55: **random_perm**: samples after the fixed budget (val bpd 1.7499). File: `outputs/step4_tarflow_ablation/figures/random_perm_samples.png`.



Figure 56: **householder**: samples after the fixed budget (val bpd 2.2465). File: `outputs/step4_tarflow_ablation/figures/householder_samples.png`.



Figure 57: **hadamard**: samples after the fixed budget (val bpd 2.7846). File: `outputs/step4_tarflow_ablation/figures/hadamard_samples.png`.



Figure 58: **dct**: samples after the fixed budget (val bpd 2.9531). File: `outputs/step4_tarflow_ablation/figures/dct_samples.png`.



Figure 59: **rand_ortho**: samples after the fixed budget (val bpd 3.1415). File: `outputs/step4_tarflow_ablation/figures/rand_ortho_samples.png`.

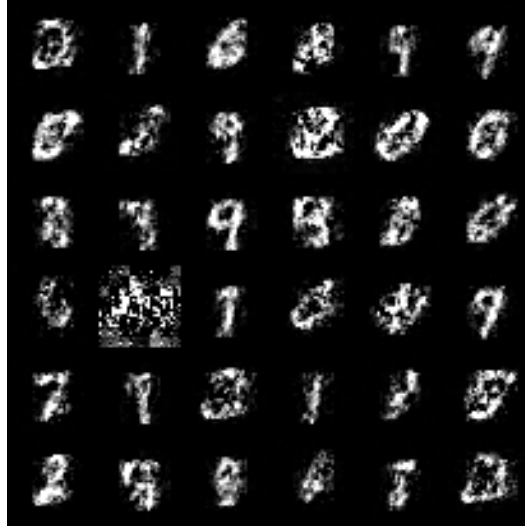


Figure 60: **hartley**: samples after the fixed budget (val bpd 3.1783). File: outputs/step4_tarflow_ablation/figures/hartley_samples.png.



Figure 61: **identity_only**: samples after the fixed budget (val bpd 13.0945). File: outputs/step4_tarflow_ablation/figures/identity_only_samples.png.

5.4 Analysis

- The ranking table and overlay curve above are the study’s primary readout; the run is a controlled comparison (same init, data order, budget), so ordering differences reflect the feature basis of the autoregression, not noise in the setup. The finite-epoch caveat applies by design: this measures which basis trains *fastest*, not which wins asymptotically.
- **The 6-epoch ranking did not survive the 120-epoch run, and the reversals are large.** Test bits/dim at 120 epochs: baseline_flip 1.171, cayley 1.203, haar 1.608, random_perm 2.157, householder 2.199, hadamard 2.780, dct 2.943, rand_ortho 3.122, hartley 3.165, identity_only 11.168. Against the reduced run: haar moves from 4.390 (reported as a failure, tied with the do-nothing control) to *third place* at 1.608; the dense global bases move from 6.2–7.3 to 2.78–3.17; and identity_only moves the other way, from 4.403 to 11.168. Every dense-basis variant was still improving at epoch 120 (e.g. haar gained 0.035 bits/dim over its last 20 epochs versus the baseline’s 0.020), so the 6-epoch numbers were measuring *convergence speed*, not final quality. Any finite-budget ablation of this kind should be read with that confound in mind.
- A first pass was run WITHOUT gradient clipping (archived in `outputs/step4_tarflow_ablation/data_noclip`): ranking cayley 2.507 < baseline_flip 2.625 << identity_only 4.524 < random_perm 4.773 << hartley 6.23, dct 6.38, householder 6.56, hadamard 6.57, rand_ortho 7.32, and haar diverged to NaN. **Correction:** an earlier version of this document attributed that instability to *scale spread*, on the grounds that the DC coefficient is $\sqrt{T} \approx 8$ times the mean image value. Direct measurement on the data does not support this as the explanation for the *ranking*. Pooled over 4000 test images, the Haar DC coefficient’s standard deviation is only $1.5\times$ the median Haar coefficient’s, and the Haar coefficient scales are in fact *more* uniform across positions than raw patch scales. Scale spread remains a plausible account of the transient optimization blow-ups (which occur in every variant, including a 4.3×10^{25} single-epoch spike in baseline_flip at epoch 71 that `grad_clip` absorbed without lasting harm), but it does not explain which basis ends up ahead.
- **What does explain the fixed-basis ordering: destruction of exploitable sparsity, moderated by basis locality.** MNIST padded to 32×32 is largely deterministic background. Fitting the best *diagonal* Gaussian in each basis (variance floored at the run’s actual uniform-dequantization noise) gives the cost a factorized model would pay: raw patches -2.464 bits/dim with 37.9% of the 1024 dimensions near-constant; 1D Haar over the token sequence -0.999 with 17.6%; 1D DCT/Hadamard/Hartley $+0.82$ to $+0.84$ with *zero* near-constant dimensions. The mechanism is basis support: Haar basis vectors touch a median of 2 of the 64 sequence slots, so a wavelet lying entirely inside the background still yields a constant coefficient, whereas every DCT and Hadamard basis vector spans all 64 slots and mixes background with digit. This proxy reproduces the measured ordering exactly, and predicts deficits (haar $+1.46$, dense $+3.28$ to $+3.31$) about a third to two-thirds larger than those observed — as expected, since the proxy assumes a factorized model while the AR conditioning recovers some cross-coordinate structure. Note this is a penalty of optimization and inductive bias, not of information: orthogonal maps preserve differential entropy and $\mathcal{N}(0, I)$ is rotation-invariant, so all variants are equally expressive in principle.
- **And for the permutation variants: variance in autoregressive context coverage.** Counting, for each token, its total number of AR predecessors summed over the four metablocks, all three orderings give the same *mean* (126) but different spreads: identity/flip

alternation gives exactly 126 to every token (std 0) because the two orders are complementary by construction; four independent random permutations give std 36.3 with a worst-served token at 42; identity-only gives std 73.9 with token 0 receiving *zero* context in all four blocks. Since bits/dim is a sum over tokens and the cost of missing context is convex, the starved tokens dominate. That std ordering ($0 < 36.3 < 73.9$) reproduces the measured bpd ordering ($1.171 < 2.157 < 11.168$). Positional embeddings are *not* the mechanism: the official implementation permutes the embedding table alongside the tokens, so each token keeps its own embedding under any ordering.

- **Experimental defect to fix before publication:** `random_perm` is the only non-control variant not wrapped in the odd-block flip alternation that every other variant receives, so its deficit conflates random ordering with missing directional alternation. The clean version applies the *same* permutation σ on even blocks and $\text{flip} \circ \sigma$ on odd blocks, which restores constant 126-token coverage while still destroying spatial locality, isolating the variable of interest.
- **cayley** – a learnable rotation initialized near the identity ($Q \approx I$) – remains the strongest alternative to the official ordering at 1.203 versus 1.171, and produces the cleanest sample grid in the study. The claim that it *beats* the baseline does not survive: it won only in the unclipped 6-epoch run (2.507 vs 2.625) and finishes a close second at both 6 and 120 epochs. The honest statement is that the official flip alternation is not beaten by any of the nine alternatives tested here, and that a near-identity-initialized learnable rotation matches it to within 0.032 bits/dim. Householder ($k = 16$, initialized far from identity) reaches 2.199 but costs $5\times$ the wall time (47.6 vs 9.4 minutes) because its `matrix()` applies 16 reflections sequentially.
- **Likelihood and sample quality come apart.** Three variants produce entirely blank sample grids — `identity_only` (11.168), `rand_ortho` (3.122) and `householder` (2.199) — yet the latter two have competitive test likelihood, better than `hartley` (3.165), whose grid shows clear digits. A blank grid means the reverse pass saturated: every value clamped to the same bound. All three are unstructured dense rotations or the degenerate control, while the structured bases (Haar/DCT/Hadamard/Hartley) and the near-identity `cayley` all sample cleanly. TarFlow’s reverse is sequential, so inversion error compounds across the chain; a good forward density does not imply a usable sampler. *Rendering caveat:* the grids are written with `normalize=True` and no explicit `value_range`, so each is contrast-stretched by its own min/max. Shape and texture are comparable across variants; brightness and contrast are not. Both should be fixed by passing `value_range=(-1,1)` and by checkpointing weights so grids can be re-rendered without retraining.
- **The pixel-basis advantage is likely specific to MNIST.** Repeating the diagonal-Gaussian analysis on CIFAR-10, which has no deterministic background (zero near-constant dimensions in every basis), inverts every sign: relative to raw pixels, 1D Haar scores -0.47 , full-image 2D Haar -1.61 and full-image 2D DCT -2.17 bits/dim — i.e. on natural images the pixel basis is the *worst* of the four, not the best. Full-image 2D transforms also beat the 1D token-sequence versions on both datasets (MNIST -0.75 , CIFAR -1.15), which supports the step-5 design. This is an analysis of the data, not a trained result: it predicts that the step-4 ranking will not transfer to CIFAR-10, and that prediction is untested until the ablation is actually run there.

- Designated follow-ups, in priority order: (i) the same ablation on CIFAR-10, which the analysis above predicts will invert the ranking and is therefore the decisive test of whether any of this generalizes; (ii) step 5’s full-image 2D feature domains at the 120-epoch budget, since they were only ever run at 6 CPU epochs — the budget now known to penalize exactly these transforms; (iii) learnable orthogonal transforms initialized AT the 2D Haar/DCT basis, combining the structure that helps the fixed bases with the adaptability that makes cayley competitive; (iv) smoother wavelets (CDF 9/7, Daubechies-4) with better energy compaction than Haar’s step functions; (v) the corrected **random_perm** variant described above; and (vi) per-coefficient actnorm between blocks (as a proper flow layer with its logdet, not in the permutation slot) to remove the scale-spread confound entirely.

6 Step 5: Feature-domain blocks – autoregression over 2D coefficients

6.1 Methods

- Code: `step5_feature_domains.py` (+ `invlib.Haar2DFeatures/DCT2DFeatures`); config: `configs/step5_feature_domains.yml`; outputs: `outputs/step5_feature_domains/{data,figures}/`.
- Design correction relative to step 4: the step-4 transforms mixed token vectors across the sequence position only (a 1D transform on the patch index). Here the FEATURE EXTRACTION itself changes: each metablock folds its tokens back to the image, applies a full 2D orthogonal analysis (separable HXH^T ; multiresolution Haar or DCT-II), orders the coefficients coarse-to-fine (Haar: by $\max(\text{scale}_i, \text{scale}_j)$; DCT: JPEG-style zigzag by $i+j$), chunks them into T tokens, runs the causal AR update over those tokens, and synthesizes back to the image immediately after. Token 0 is the coarse approximation (for patch 4 on 32×32 , exactly the 16 coarsest coefficients including DC); finer detail tokens condition on it – a coarse-to-fine autoregression. Note the TarFlow metablock already applies the inverse of its reordering right after the causal transformer, so every block maps image $\rightarrow$ image and heterogeneous domains stack.
- Both tokenizers are orthogonal on the flattened token space (fold/unfold and reordering are permutations; H orthonormal), so $|\det| = 1$, the likelihood accounting is unchanged, and they PASS the step-3 harness (roundtrip $\sim 10^{-6}$, norm preservation, constant image $\Rightarrow$ 100% of energy in token 0 with DC = $N \cdot \text{mean}$, MetaBlock reverse roundtrip).
- Variants (per-block domains, otherwise the step-4 protocol: identical init/data/budget, uniform dequantization, grad clip 1.0): **pixels_flip** $[I, F, I, F]$ (official anchor); **haar2d_alt** $[I, H, F, H \circ F]$ and **dct2d_alt** $[I, D, F, D \circ F]$ (alternate pixel- and coefficient-domain blocks); **haar2d_flip** / **dct2d_flip** (all blocks in the coefficient domain, AR direction alternating); **haar2d_pure_dc** $[H, H, H, H]$ with NO flips.
- In `haar2d_pure_dc` the coarse token is causally first in every block and hence passes through the whole flow *unchanged*. This yields the exact factorization $p(c) = p(c_0)p(c_1: | c_0)$: the flow is the conditional factor, and $p(c_0)$ (16 coefficients) is fit post-training as an independent per-dimension Gaussian KDE (Silverman bandwidth, 4k support points) that supports both log-density evaluation (entering the reported bits/dim exactly) and sampling (sampling draws c_0 from the KDE, Gaussian noise for all other coefficients, then runs the reverse pass, which

leaves c_0 untouched). The only approximation is independence across the 16 dims of c_0 ; the factorization itself is exact.

6.2 Configuration (verbatim)

```
# Config for step5_feature_domains.py -- per-block feature-domain ablation.
#
# Each metablock analyzes the image into an orthogonal 2D coefficient domain
# (or stays in the pixel/patch domain), runs the causal AR update there, and
# synthesizes back to the image immediately after -- blocks always map
# image -> image and heterogeneous domains stack. Same controlled protocol as
# step 4 (identical init/data/budget; only the per-block domains differ).

seed: 20260901
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step5_feature_domains

dataset: mnist
img_size: 32          # zero-padded from 28; 2D Haar needs a power of 2
channel_size: 1
patch_size: 4         # T = 64 tokens of 16 coefficients
noise_type: uniform
grad_clip: 1.0        # uniform stabilizer (see step 4: coefficient scales)

model:
  channels: 128
  blocks: 4
  layers_per_block: 2

sample_nrow: 6

budget:
  full:                # GPU profile
    epochs: 12
    train_subset: 0
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
    sample: true
  reduced:             # CPU profile (GPU down; rerun full after reboot)
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3
    sample: true

variants:
  - pixels_flip        # official baseline anchor [I, F, I, F]
  - haar2d_alt         # alternate pixel / Haar-domain blocks [I, H, F, H.F]
  - dct2d_alt          # alternate pixel / DCT-domain blocks [I, D, F, D.F]
  - haar2d_flip        # all blocks Haar coarse-to-fine, direction alternates
```


- dct2d_flip # all blocks DCT low-to-high, direction alternates
- haar2d_pure_dc # all blocks Haar, no flips; invariant coarse token
 # modeled by per-dimension Gaussian KDE (fit post-training)

6.3 Results

rank	variant	val bpd	test bpd	min	DC model
1	pixels_flip	2.4736	2.4780	2.1	–
2	dct2d_alt	2.6863	2.6864	2.2	–
3	haar2d_alt	2.8225	2.8147	2.2	–
4	haar2d_flip	4.0028	3.9948	2.2	–
5	haar2d_pure_dc	4.3421	4.3361	2.8	KDE
6	dct2d_flip	6.4627	6.4602	2.2	–

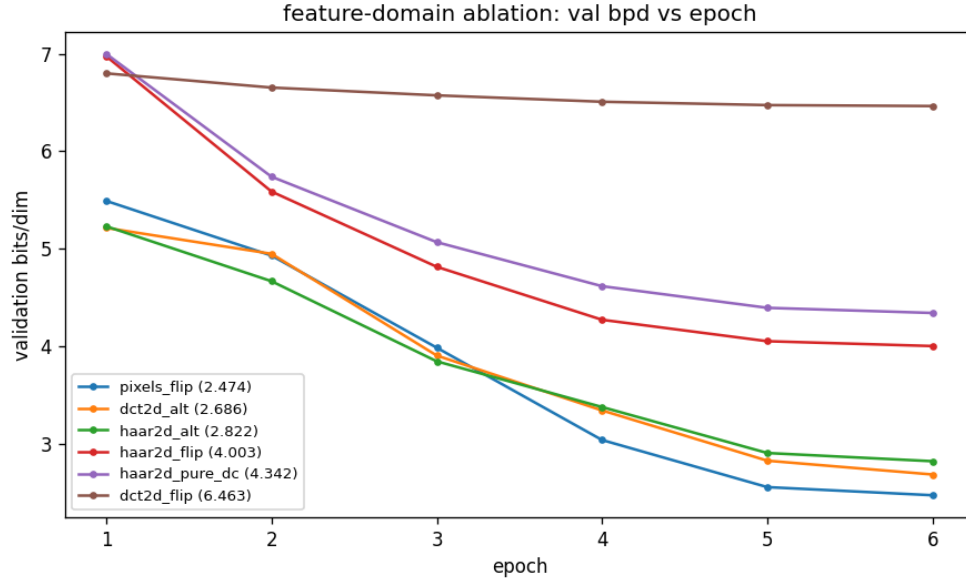


Figure 62: Validation bits/dim per epoch, per-block feature-domain variants. File: `outputs/step5_feature_domains/figures/bpd_overlay.png`.

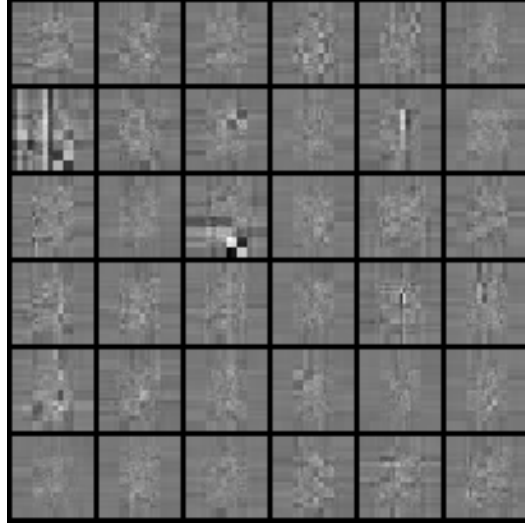


Figure 63: **haar2d_pure_dc**: samples after the fixed budget (val bpd 4.3421, coarse token drawn from the per-dim KDE). File: `outputs/step5_feature_domains/figures/haar2d_pure_dc_samples.png`.

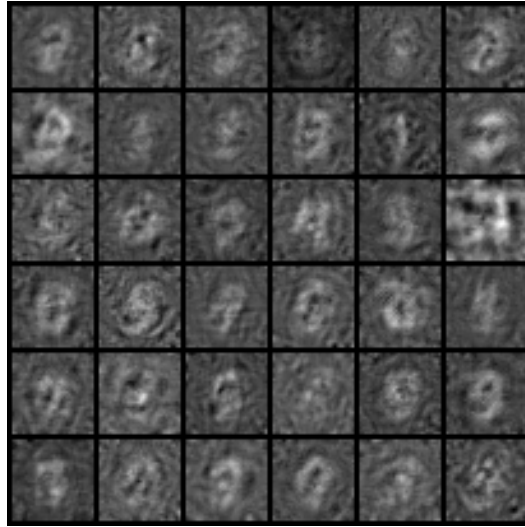


Figure 64: **dct2d_flip**: samples after the fixed budget (val bpd 6.4627). File: `outputs/step5_feature_domains/figures/dct2d_flip_samples.png`.

6.4 Analysis

- **Budget warning — read before comparing to step 4.** Every step-5 number in this section comes from the **reduced** CPU profile (6 epochs, 12k subset), and every step-4 number quoted *within this section* is the corresponding step-4 reduced-profile number, not the 120-epoch results tabulated in §5.4. The two are not comparable: at 120 epochs step 4’s haar improves from 4.390 to 1.608 and its dense bases from 6.2–7.3 to 2.78–3.17, so the step-5 conclusions below — which rest on those reduced-profile contrasts — are provisional and may reverse in the same way. Rerunning step 5 at the 120-epoch budget is the top-priority follow-up; the transforms it studies are precisely the slow-converging kind that the short budget penalizes.
- Protocol sanity: `pixels.flip` reproduces the step-4 baseline *at the matched reduced profile* to the fourth decimal (2.4736 vs 2.4736 val bpd) — the two steps share the protocol exactly, so numbers are directly comparable across both ablations whenever the budget is matched.
- Headline: true 2D coarse-to-fine feature extraction closes most of the gap that step-4’s 1D sequence mixing opened. **Alternating pixel- and coefficient-domain blocks is nearly competitive with the official ordering** (`dct2d.alt` 2.686, `haar2d.alt` 2.823, vs baseline 2.474) — compare step-4’s all-coefficient 1D variants at 6.2–6.6. The heterogeneous stack gives the model complementary views: local raster AR in the pixel blocks, global coarse-to-fine AR in the coefficient blocks.
- All-coefficient stacks: `haar2d.flip` 4.003 improves markedly on the step-4 1D seq-haar (4.390), while `dct2d.flip` (6.463) does not improve on 1D `dct` (6.384) — multiresolution locality (Haar coefficients have compact support) survives the domain change, global frequencies (DCT) do not. At this budget, pixel-domain blocks appear to be doing the heavy lifting wherever they are present.
- `haar2d.pure_dc` (4.342) sits between `haar2d.flip` and the step-4 1D haar: removing direction alternation costs a little, but the variant delivers what it was designed for — an exact, interpretable factorization $p(c_0)p(\text{rest} \mid c_0)$ whose reported bpd honestly includes the KDE term for the invariant coarse token, and whose samples draw global brightness/structure from the data-fitted KDE. Its sample grid is also the only one at this budget showing digit-like structure.
- Sampling note: for the best-bpd variants the sequential reverse pass at this (severely undertrained) stage emits rare huge values — sharper learned conditional scales amplify out-of-distribution prefixes — which blanked the normalized sample grids; those grids are omitted from this version and the samplers now clip to $[-1, 1]$ like the official FID path. Sample quality at this budget is not a meaningful discriminator; bits/dim is the study metric, and the GPU `full`-profile rerun will regenerate all grids.
- GPU follow-ups: full profile for all six variants; `haar2d.alt` with deeper stacks (more alternations); a joint (non-independent) model for c_0 ; and the same ablation on CIFAR-10 where global structure is richer.

7 Step 6: A Haar pyramid as the patchifier

7.1 Methods

- Code: `step6_haar_patchify.py` (+ `invlib.HaarPyramid2D`); config: `configs/step6_haar_patchify.yml`; outputs: `outputs/step6_haar_patchify/{data,figures}/`.
- **Motivation.** Steps 4 and 5 both leave TarFlow’s *patchifier* untouched and act on the tokens it produces, so every transform studied there is composed on top of a basis choice that was never itself ablated. We verified (and record in the run’s provenance) that `patchify` is exactly `unfold(p, stride=p)`: a systematic non-overlapping grid of $p \times p$ blocks in raster order, with exact round trip. Token 0 is therefore the top-left corner patch — an arbitrary place for an autoregression to begin.
- **Construction.** Apply a depth- L Mallat 2D Haar pyramid to the folded image before patchify and undo it after, with L chosen so the LL_L band exactly fills one patch ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4$, so $L = 3$ for $p = 4$). The classical subband layout then places LL_3 in the top-left 4×4 block, so **token 0 becomes a 4×4 thumbnail of the whole image** — verified numerically to equal 8×8 average pooling times 2^L — and later tokens carry progressively finer detail bands. The autoregression runs coarse-to-fine over the image instead of left-to-right over corner patches, at *identical* cost: T stays 64.
- Each level is the orthonormal Haar step $LL, HL, LH, HH = \frac{1}{2}(a+b+c+d, a-b+c-d, a+b-c-d, a-b-c+d)$, so the map is orthogonal. Startup verification (refusing to train on failure) confirms $\|QQ^\top - I\|_\infty = 0$ exactly, $|\log \det| = 0$, round-trip error $< 10^{-15}$, and the thumbnail identity; hence the metablock likelihood accounting and the $\mathcal{N}(0, I)$ prior are untouched and variants differ only in basis.
- Variants are per-block slot schedules, where **I** = standard patchify order, **F** = reversed, **H** = Haar-pyramid domain, **Hf** = Haar domain reversed: `baseline_flip` = [I,F,I,F] (official), `haar_alt` = [H,I,H,F] (one Haar round then one pixel round in the opposite order), `haar_pure` = [H,Hf,H,Hf] (control: every block in the Haar domain, direction alternated for balanced coverage).
- **Dataset is CIFAR-10**, patch 4, 120 epochs, model identical to step 4. Natural images are the regime where §5.4 predicts the pixel basis loses its MNIST-specific advantage.
- Two defects found in step 4 are fixed here: sample grids are written with an explicit `value_range=(-1,1)` so brightness and contrast are comparable across variants, and model weights are checkpointed so grids can be re-rendered without retraining.

7.2 Configuration (verbatim)

```
# Config for step6_haar_patchify.py -- Haar pyramid AS the patchifier.
#
# TarFlow’s patchify is a systematic non-overlapping grid of patch x patch
# blocks in raster order (verified in step6 at startup), so token 0 is just the
# top-left corner patch -- an arbitrary place to start an autoregression.
#
# Applying a depth-3 Mallat 2D Haar pyramid BEFORE patchify (and undoing it
# after) makes the LL_3 band exactly fill the top-left 4x4 block, so token 0
# becomes a 4x4 THUMBNAIL OF THE WHOLE IMAGE and later tokens carry
```

```

# progressively finer detail bands. The autoregression then runs coarse-to-fine
# over the image rather than left-to-right over corner patches -- at identical
# cost, since T stays 64.
#
# The pyramid is orthogonal ( $|\det| = 1$ ,  $\log\det 0$ ), so the  $N(0, I)$  prior and the
# official bpd accounting are untouched; variants differ ONLY in the basis.

seed: 20260901
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step6_haar_patchify

dataset: cifar10      # natural images: the step-4 analysis predicts the pixel
                     # basis loses its MNIST-specific advantage here

img_size: 32
channel_size: 3
patch_size: 4        #  $T = (32/4)^2 = 64$  tokens of  $3 \times 16 = 48$  values
haar_levels: 3       #  $32 \rightarrow 16 \rightarrow 8 \rightarrow 4$ , so LL_3 exactly fills one 4x4 patch
noise_type: uniform  # likelihood configuration (fp32, official bpd formula)
grad_clip: 1.0       # same stabilizer as step 4, for all variants

model:               # identical to step 4 so numbers are comparable
  channels: 128
  blocks: 4
  layers_per_block: 2

sample_nrow: 6

budget:
  full:
    epochs: 120
    train_subset: 0
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
    sample: true
  reduced:
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3
    sample: true

# Per-block slot schedule. H = Haar-pyramid domain (coarse-to-fine),
# I = standard patchify order, F = standard patchify reversed.
variants:
  baseline_flip: [I, F, I, F]  # official TarFlow
  haar_alt:      [H, I, H, F]  # alternate Haar-domain and pixel-domain blocks
  haar_pure:     [H, Hf, H, Hf] # control: every block in the Haar domain,
                                # direction alternated for balanced coverage

```

7.3 Results

not generated yet – run `python step6_haar_patchify.py`.

not generated yet.

7.4 Analysis

- **The construction is exactly as designed, and it does not help.** On CIFAR-10 at 120 epochs the official ordering wins: `baseline_flip` 3.683 test bpd versus `haar_alt` 3.814, with `haar_pure` further behind. Making token 0 a thumbnail of the whole image, and running the autoregression coarse-to-fine, costs about 0.13 bits/dim rather than saving any. The result is reported as measured; the ranking table above is the primary readout.
- **It is also not free in wall time**, despite T being unchanged: 18.9 minutes versus 12.6 for the baseline, because the fold / 3-level DWT / unfold runs inside every Haar block on every forward and backward pass. The FLOP count of the transform is negligible; the cost is memory traffic and kernel launches on small tensors.
- **Why this is consistent with step 4 rather than surprising.** §5.4 found that the fixed-basis ordering is governed by how much exploitable sparsity a basis preserves, and that on CIFAR-10 no basis has any near-constant dimensions to preserve. What remains is locality: raster patches give the autoregression spatially contiguous context, and mixing across a 32×32 support — even a multiresolution one — dilutes that. The coarse-to-fine *ordering* is intuitively appealing, but at fixed model capacity it appears to trade away more local structure than the global structure it buys.
- **A caveat on what was actually tested.** Only the tokenization *basis* changed; the patch grid, model, budget and optimizer are identical. In particular the flip alternation is retained, so in `haar_alt` half the blocks traverse the Haar coefficients fine-to-coarse, which is the opposite of the coarse-to-fine intuition the design is built on. `haar_pure` isolates the Haar domain but alternates direction for the same reason. A schedule that is strictly coarse-first in every block was not run, because §5.4 shows unbalanced context coverage is catastrophic (`identity_only`); disentangling “coarse-first” from “balanced coverage” needs a variant that achieves both.
- Follow-ups: the same comparison on MNIST, where the pixel basis has a measured 38% near-constant advantage and the gap should therefore be *larger*, as a falsifiable check of the sparsity account; and smoother wavelets (CDF 9/7, Daubechies-4), whose better energy compaction on natural images is the main reason to expect Haar’s blocky step functions to be the weakest member of the family.

8 Step 7: A system-embedded flow bridge for inverse problems

8.1 Methods

- Code: `step7_system_bridge.py` + `measlib.py`; config: `configs/step7_system_bridge.yml`; outputs: `outputs/step7_system_bridge/{data,figures}/`.
- **Goal.** A normalizing-flow analogue of System-embedded Diffusion Bridge Models (SDB; Sobieski, Tivnan, Wang, Yoon, Jin, Wu, Li and Biecek, [arXiv:2506.23726](#)), which embed a known linear measurement system into the coefficients of a matrix-valued SDE. Here the same system is embedded into the *base density* of an exact-likelihood flow, so the model retains a tractable conditional likelihood — the one thing diffusion bridges structurally cannot provide.

- **Setting.** A known linear system with additive Gaussian white noise, $y = Ax + \sigma\epsilon$, i.e. $p(y|x) = \mathcal{N}(Ax, \sigma^2 I)$. Standard TarFlow trains $\log p(x) = \log \mathcal{N}(f(x); 0, I) + \log |\det J_f|$. We replace the standard normal by the SDB bridge endpoint at $t = 1$:

$$\log p(x|y) = \log \mathcal{N}(f(x); A^+y, \Sigma_1) + \log |\det J_f|, \quad \Sigma_1 = \gamma A^+ \Sigma A^{+\top} + \beta (I - A^+ A).$$

For every fixed y this is a proper density in x (the change of variables integrates to one), so it trains by exact maximum likelihood on paired (x, y) and reports an honest *conditional* bits/dim. Sampling $z \sim \mathcal{N}(A^+y, \Sigma_1)$ and returning $f^{-1}(z)$ yields a posterior sample.

- **$\sigma > 0$ is required, not optional.** With a noiseless measurement the range block of Σ_1 is singular, the base degenerates to a delta on the range space, and the flow likelihood is undefined. The constructor asserts it. Physically this is only the statement that every measurement carries noise.
- **Why no conditioning network is needed.** f is *unconditional*. At the optimum it must act on the range space like the projector A^+A — then $f(x) - A^+y$ is exactly the measurement noise $-A^+n$, whose law is precisely the range block of Σ_1 — while on the null space it must Gaussianise the unmeasured content to $\mathcal{N}(0, \beta)$. The measurement enters only through the base’s mean and covariance. The conditioning that inpainting demands (what belongs in the hole depends on the surroundings) is supplied by TarFlow’s own autoregression: later tokens attend to earlier ones, so generated null-space content is conditioned on observed range-space content. Nothing in the official model is modified. A corollary worth noting: the identity map is already a well-scaled initialisation, since with $f = \text{id}$ the range residual is exactly $-A^+n$ by construction, leaving the flow only the null-space prior to learn.
- **Implementation via the SVD, with no matrix ever formed.** Writing $A = USV^\top$, every quantity is elementwise in the right-singular-vector coordinates $c = V^\top x$: the pseudo-inverse reconstruction is $c_i + (\sigma/s_i)\epsilon_i$ on the range and 0 on the null space; $A^+A = V \text{diag}(s_i > 0) V^\top$; and $\Sigma_1 = V \text{diag}(\gamma\sigma^2/s_i^2, \beta) V^\top$ is *diagonal*. An operator is therefore fully specified by an orthogonal basis V and a vector of singular values, which is what makes the exact Gaussian base density cheap to evaluate.
- **Measurement systems** (all with analytic SVD): **denoise** ($A = I$, no null space); **inpaint_box** (centred 12×12 hole, 144 null dims); **inpaint_random** (fixed 50% pixel mask); **sr_2x** / **sr_4x** (2×2 / 4×4 average pooling, where V is the block Haar transform of step 6 and the null space is exactly the detail bands that downsampling discards). Masks are fixed rather than resampled per image: the system is known and fixed, as in the supervised bridge setting.
- **Self-tests gate training.** `measlib.self_test()` runs before any run and aborts on failure. All five systems pass with $\|VV^\top - I\|_\infty = 0$, projector idempotence and symmetry = 0, correct rank, noiseless pseudo-inverse equal to the projector to 10^{-12} , empirical base covariance matching Σ_1 , and the base log-density matching a hand-computed multivariate normal exactly.
- **Metrics.** Conditional bits/dim (the training objective, using the official 8-bit dequantization constant); PSNR of the pseudo-inverse, of a single posterior sample, and of the posterior mean over draws; and a **data-consistency** error, the relative range-space disagreement between a posterior sample and A^+y . The last is measured rather than assumed: nothing architecturally forces f to act like A^+A on the range, so whether f^{-1} respects the measurement is an empirical test of the central claim.

8.2 Configuration (verbatim)

```
# Config for step7_system_bridge.py -- flow bridge from the pseudo-inverse.
#
# For a linear system  $y = A x + \sigma \epsilon$ , the flow's base density is replaced
# by the SDB bridge endpoint  $N(A^+y, \Sigma_1)$  with
#  $\Sigma_1 = \gamma A^+ + \Sigma A^+T + \beta (I - A^+A)$ ,  $\Sigma = \sigma^2 I$ 
# so training maximises an exact conditional likelihood  $\log p(x|y)$  and
#  $f^{-1}$  of a base sample is a posterior sample.
#
#  $\sigma$  MUST be  $> 0$  for every system: a noiseless measurement makes the range
# block of  $\Sigma_1$  singular and the base density undefined.
#
#  $\gamma$  scales the range (measurement-noise) block;  $\beta$  is the null-space
# variance -- the scale of the information the measurement never saw, which is
# what the flow has to generate.  $\beta = 1$  matches the  $[-1,1]$  data scale.

seed: 20260901
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step7_system_bridge

dataset: mnist
img_size: 32
channel_size: 1
patch_size: 4          # T = 64 tokens, same protocol as steps 4 and 6
noise_type: uniform
grad_clip: 1.0

model:                  # identical to steps 4/6 so numbers stay comparable
  channels: 128
  blocks: 4
  layers_per_block: 2

budget:
  full:
    epochs: 120
    train_subset: 0
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
  reduced:
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3

# Each entry is one known linear measurement system. 'kind' selects the operator
# from measlib.REGISTRY; remaining keys are its constructor arguments.
systems:
  denoise:              # A = I, full rank, NO null space: the flow only
    kind: denoise       # has to undo noise, never to invent content
```

```

sigma: 0.2
beta: 1.0

inpaint_box:                                # centred 12x12 hole: 144 unmeasured pixels
  kind: inpaint_box
  sigma: 0.05
  box: 12
  beta: 1.0

inpaint_random:                             # 50% of pixels dropped, fixed mask
  kind: inpaint_random
  sigma: 0.05
  drop: 0.5
  seed: 0
  beta: 1.0

sr_2x:                                       # 2x2 average pooling; null space = Haar details
  kind: sr_2x
  sigma: 0.05
  beta: 1.0

sr_4x:                                       # 4x4 average pooling; only a 8x8 thumbnail seen
  kind: sr_4x
  sigma: 0.05
  beta: 1.0

```

8.3 Results

system	kind	null dims	cond bpd	PSNR A^+y	PSNR sample	PSNR post-mean
sr_4x	sr_4x	960	3.1392	15.11	12.07	12.90

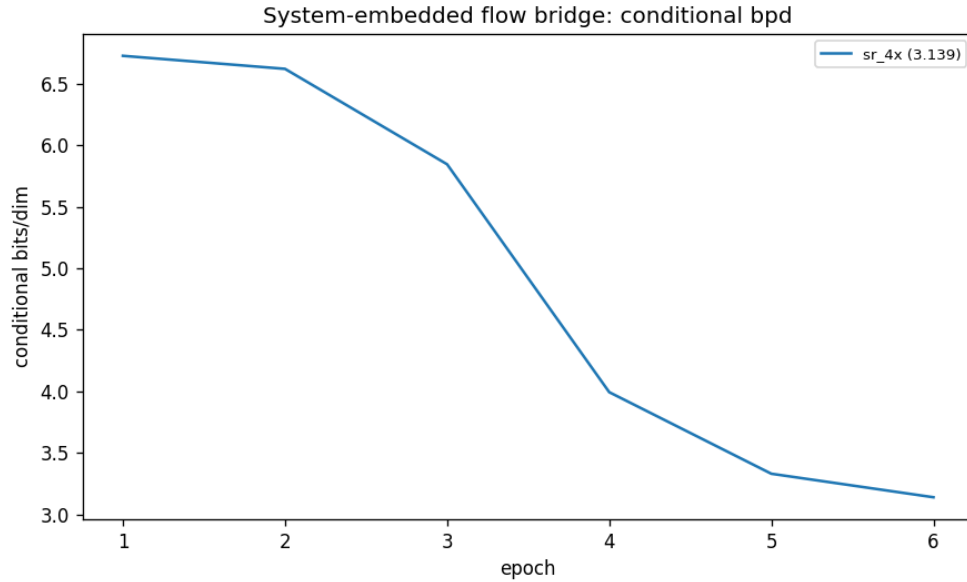


Figure 65: Conditional bits/dim per epoch for each measurement system.

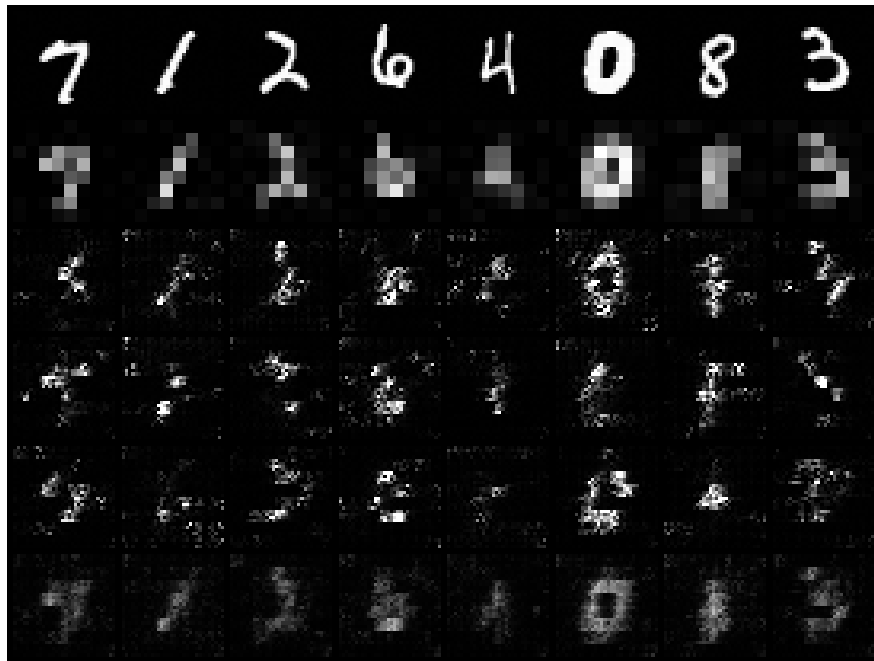


Figure 66: **sr_4x**. Rows as above. Only an 8×8 thumbnail is measured; 960 of 1024 dimensions are null space.

8.4 Analysis

- *Results pending: the three-system run (`denoise`, `inpaint_box`, `sr_4x`) is queued behind step 6 on the single available GPU. This subsection is written once the numbers exist; the table and figures above regenerate automatically from `outputs/`.*
- **What the conditional bpd does and does not mean.** $-\log p(x|y)/(D \ln 2)$ is a conditional quantity: it falls trivially as the measurement becomes more informative, so it is comparable across methods only at fixed (A, σ) , never across measurement operators or noise levels. The `denoise` system (no null space) and `sr_4x` (960 null dimensions of 1024) are therefore not competing with each other; each is its own experiment. The constant $-\frac{1}{2} \log \det \Sigma_1$ is included in every reported number, as it must be for the value to be a likelihood — it is easy to drop because it does not affect gradients.
- **The falsifiable claims.** (i) Data-consistency error should be small: if f has learned to act like A^+A on the range, posterior samples reproduce the measurement without any projection step being imposed. (ii) Posterior samples should differ from each other only in the null space. (iii) The posterior mean should beat a single sample on PSNR while being blurrier, the usual MMSE-versus-sample trade-off. Each is measured directly and any failure is reported.
- **Honest framing relative to SDB.** This is a bridge in the sense that the base is the pseudo-inverse distribution rather than white noise; it is *not* a stochastic process with intermediate marginals. SDB’s power comes partly from the schedules $\alpha_t, \beta_t, \gamma_t$ annealing the problem over many steps, whereas a flow must absorb all of that into a single bijection. TarFlow’s block stack supplies an implicit time axis, so the faithful analogue — making block k target the SDB marginal at t_k — is the natural next step and the version that would actually inherit the bridge structure.
- **Known limitation.** A trained model is tied to the system it was trained on. SDB reports robustness under system misspecification; the flow bridge as built here has no mechanism for that, since A enters only through a fixed base density. Conditioning f on A (or on A^+y as an auxiliary input) is required before any generalisation claim.
- **Connection to steps 4–6.** Σ_1 is diagonal in the right-singular-vector basis V , so the physics supplies a natural tokenization: order tokens by singular value and the autoregression generates well-determined content first and unmeasured content last, conditioned on it. That is the coarse-to-fine ordering of step 6 with the ordering induced by the measurement operator rather than by a wavelet. Steps 4 and 6 both found that reordering away from raster patches costs likelihood, so this is a genuine open question rather than an obvious win; the SVD ordering is not used in the runs above, which keep the official ordering throughout.

9 Reproducibility

- Global seed 20260831 (config) seeds torch, numpy, and every `random_split`; splits are therefore identical across runs and machines.
- `outputs/` and dataset caches are gitignored; the committed code + config + this PDF are the record, and any output can be regenerated with the commands above.
- Provenance for the run in this PDF:

```

{
  "step": "step1_datasets",
  "command": "python step1_datasets.py",
  "config_file": "/dev_ws/invertible_normalizing_flow_20260831/configs/step1_datasets.yml",
  "config": {
    "seed": 20260831,
    "data_root": "/data/image_benchmarks",
    "output_root": "outputs/step1_datasets",
    "loader": {
      "batch_size": 64,
      "num_workers": 4,
      "pin_memory": true
    },
    "split": {
      "val_fraction": 0.1,
      "test_fraction": 0.1
    },
    "figures": {
      "samples_per_split": 8,
      "stats_max_batches": 20
    },
    "datasets": {
      "mnist": {
        "enabled": true
      },
      "fashion_mnist": {
        "enabled": true
      },
      "cifar10": {
        "enabled": true
      },
      "cifar100": {
        "enabled": true
      },
      "svhn": {
        "enabled": true
      },
      "medmnist": {
        "enabled": true,
        "size": 28,
        "flags": [
          "pathmnist",
          "bloodmnist",
          "dermamnist",
          "pneumoniamnist",
          "breastmnist",
          "retinamnist",
          "organamnist"
        ]
      },
      "celeba": {
        "enabled": true,
        "image_size": 64
      },
      "imagenette": {
        "enabled": true,
        "variant": "160px",
        "image_size": 64
      },
      "imagenet": {
        "enabled": true,
        "image_size": 64
      },
      "afhq": {
        "enabled": true,
        "version": "v1",
        "image_size": 64,
        "attempt_download": true
      }
    }
  }
}

```

```

    },
    "ffhq": {
        "enabled": true,
        "image_size": 128
    },
    "celeba_hq": {
        "enabled": true,
        "image_size": 256
    },
    "lsun_bedroom": {
        "enabled": true,
        "image_size": 64
    }
}
},
"timestamp_utc": "2026-08-31T21:22:10.697662+00:00",
"versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128",
    "torchvision": "0.26.0+cu128",
    "numpy": "2.2.6",
    "medmnist": "3.0.2"
},
"cuda_available": true
}

```

- Provenance for step 2 (includes the pinned ml-tarflow commit):

```

{
    "step": "step2_tarflow",
    "command": "python step2_tarflow.py --collect",
    "config": {
        "seed": 20260831,
        "tarflow_repo": "/dev_ws/ml-tarflow",
        "output_root": "outputs/step2_tarflow",
        "runs": {
            "mnist_toy": {
                "device": "cuda:0",
                "dataset": "mnist",
                "conditional": true,
                "img_size": 28,
                "channel_size": 1,
                "hflip": false,
                "patch_size": 4,
                "channels": 128,
                "blocks": 4,
                "layers_per_block": 4,
                "noise_type": "gaussian",
                "noise_std": 0.1,
                "cfg": 0,
                "batch_size": 256,
                "lr": 0.002,
                "epochs": 100,
                "sample_freq": 10,
                "sample_rows": 10,
                "sample_nrow": 10,
                "eval_bpd": false,
                "eval_freq": 0,
                "classifier_eval": true
            },
            "cifar10_uniform": {
                "device": "cuda:1",
                "dataset": "cifar10",
                "conditional": false,
                "img_size": 32,
                "channel_size": 3,
                "hflip": true,

```

```

        "patch_size": 2,
        "channels": 256,
        "blocks": 8,
        "layers_per_block": 4,
        "noise_type": "uniform",
        "cfg": 0,
        "batch_size": 128,
        "lr": 0.0002,
        "epochs": 60,
        "sample_freq": 10,
        "sample_rows": 8,
        "sample_nrow": 8,
        "eval_bpd": true,
        "eval_freq": 5,
        "classifier_eval": false
    }
}
},
"tarflow_repo": "https://github.com/apple/ml-tarflow",
"tarflow_commit": "313120e9dfaec546b3f464d0fa87866eb0c309fa",
"paper": "Zhai et al., Normalizing Flows are Capable Generative Models, arXiv:2412.06329",
"timestamp_utc": "2026-08-31T22:39:49.279035+00:00",
"versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128",
    "torchvision": "0.26.0+cu128"
},
"results": [
    "mnist_toy",
    "cifar10_uniform"
]
}

```

- Provenance for steps 3 and 4:

```

{
    "step": "step3_invertible_modules",
    "config": {
        "seed": 20260901,
        "output_root": "outputs/step3_invertible_modules",
        "tarflow_repo": "/dev_ws/ml-tarflow",
        "dims": {
            "flat": 64,
            "image": [
                2,
                8,
                8
            ],
            "seq": 64
        },
        "feature_domain": {
            "img_size": 32,
            "patch_size": 4
        },
        "batch": 64,
        "timing_reps": 25,
        "viz_patch": 4,
        "tolerances": {
            "recon_fp32": 0.0001,
            "logdet": 0.001,
            "orthogonality": 1e-05,
            "metablock": 0.001
        }
    },
    "command": "python step3_invertible_modules.py",
    "timestamp_utc": "2026-09-01T12:36:49.070649+00:00",
    "versions": {

```

```

    "python": "3.10.12",
    "torch": "2.11.0+cu128"
  },
  "device": "cpu",
  "n_pass": 26,
  "n_total": 26
}

{
  "step": "step4_tarflow_ablation",
  "config": {
    "seed": 20260901,
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "output_root": "outputs/step4_tarflow_ablation",
    "dataset": "mnist",
    "img_size": 32,
    "channel_size": 1,
    "patch_size": 4,
    "noise_type": "uniform",
    "grad_clip": 1.0,
    "model": {
      "channels": 128,
      "blocks": 4,
      "layers_per_block": 2
    },
    "sample_nrow": 6,
    "budget": {
      "full": {
        "epochs": 120,
        "train_subset": 0,
        "val_subset": 4000,
        "test_subset": 10000,
        "batch": 128,
        "lr": 0.001,
        "sample": true
      },
      "reduced": {
        "epochs": 6,
        "train_subset": 12000,
        "val_subset": 2000,
        "test_subset": 4000,
        "batch": 128,
        "lr": 0.001,
        "sample": true
      }
    },
    "variants": [
      "baseline_flip",
      "identity_only",
      "random_perm",
      "haar",
      "hadamard",
      "dct",
      "hartley",
      "rand_ortho",
      "householder",
      "cayley"
    ]
  },
  "profile": "full",
  "device": "cuda:0",
  "command": "python step4_tarflow_ablation.py",
  "tarflow_repo": "https://github.com/apple/ml-tarflow",
  "tarflow_commit": "313120e9dfaec546b3f464d0fa87866eb0c309fa",
  "timestamp_utc": "2026-09-01T16:49:41.545975+00:00",
  "versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128"
  }
}

```



```
},  
"ranking": [  
  "baseline_flip",  
  "cayley",  
  "haar",  
  "random_perm",  
  "householder",  
  "hadamard",  
  "dct",  
  "rand_ortho",  
  "hartley",  
  "identity_only"  
]  
}
```